

Flash Attention

<https://arxiv.org/pdf/2205.14135>

If you don't remember anything, remember this:

1. Flash Attention is *simply an efficient implementation* of the attention operation

If you don't remember anything, remember this:

1. Flash Attention is *simply an efficient implementation* of the attention operation
 - a. Do computations in an I/O aware manner on GPUs

Incredible speedups

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FLASHATTENTION	37.6	63.9	81.4	43.5	72.7	59.8	2.4×

Outline

1. The basics
2. FlashAttention forward pass
3. FlashAttention backward pass
4. Brief notes on FlashAttention-2+3

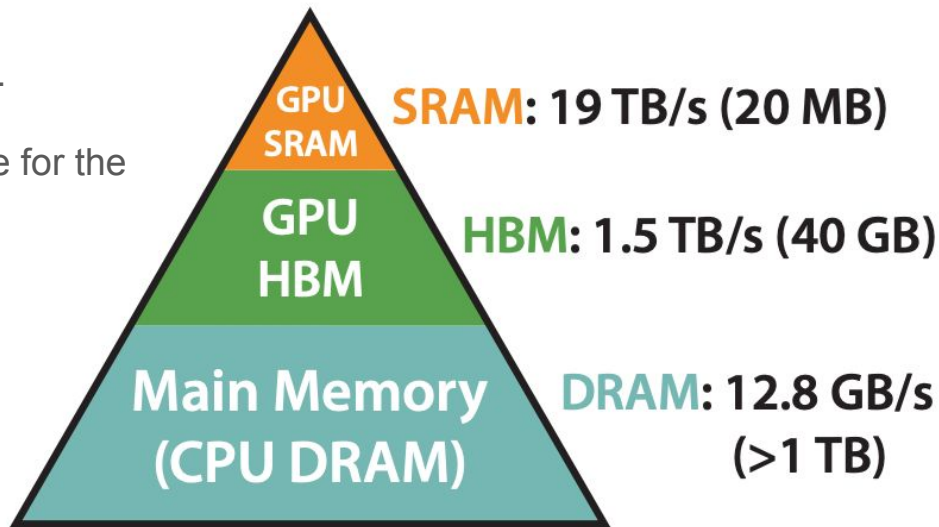
Outline

1. The basics
2. FlashAttention forward pass
3. FlashAttention backward pass
4. Brief notes on FlashAttention-2+3

Memory hierarchy - How to think about it?

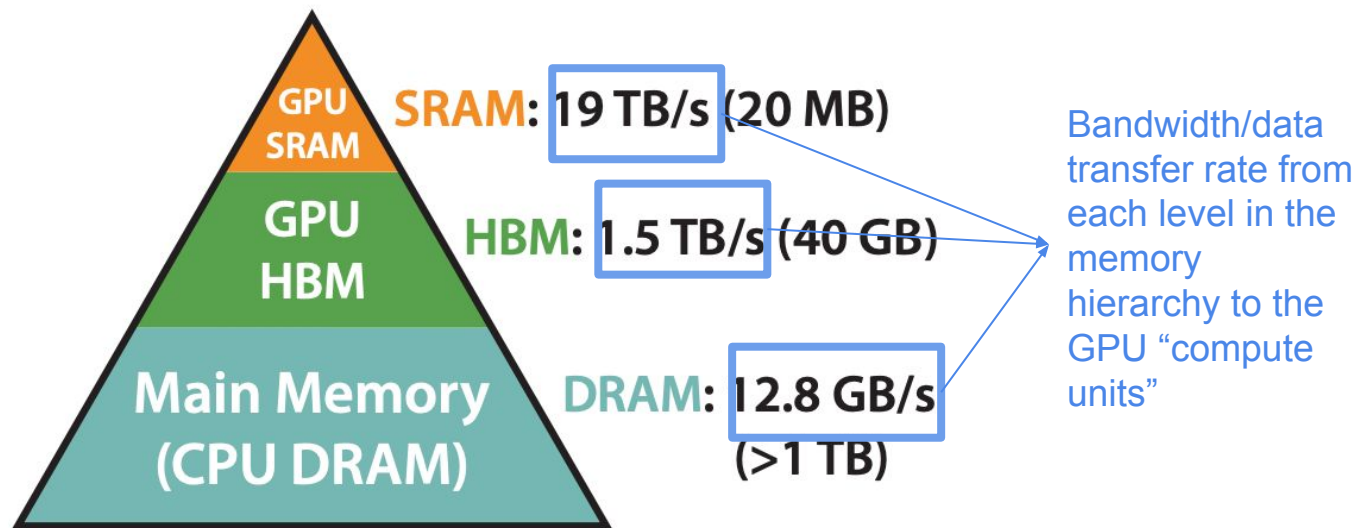
Image from original
[FlashAttention paper](#).

Reference figures are for the
[A100 GPU](#)



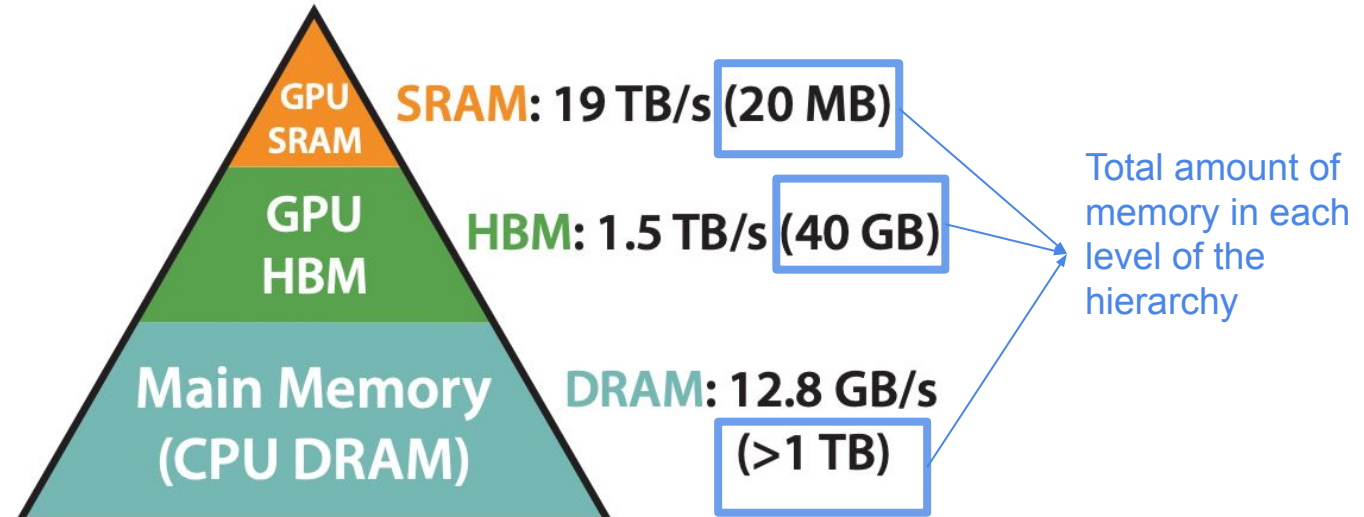
**Memory Hierarchy with
Bandwidth & Memory Size**

Memory hierarchy - How to think about it?



**Memory Hierarchy with
Bandwidth & Memory Size**

Memory hierarchy - How to think about it?

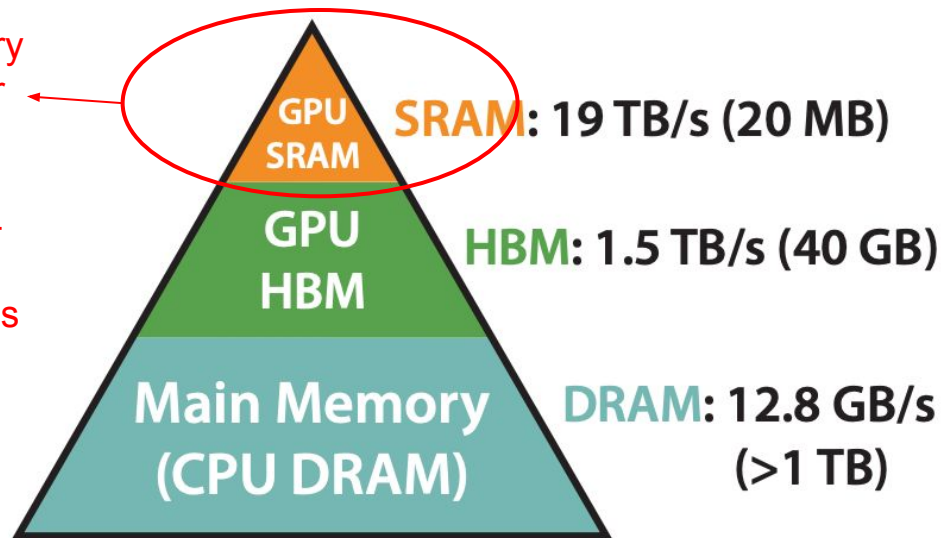


**Memory Hierarchy with
Bandwidth & Memory Size**

Memory hierarchy

Static Random Access Memory (SRAM) - closest to the tensor cores of a NVIDIA GPU.

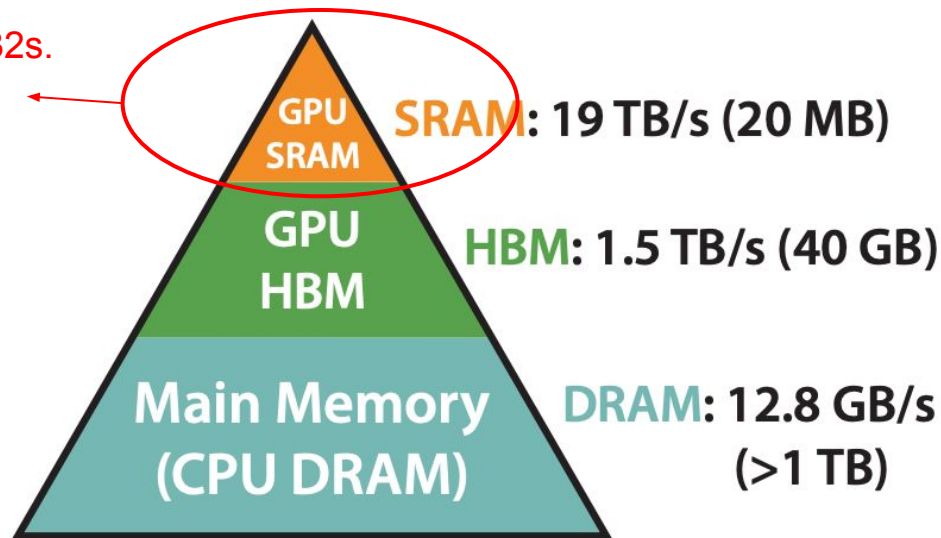
VERY FAST memory access - data stored here can be accessed very fast, but there is very little of it



**Memory Hierarchy with
Bandwidth & Memory Size**

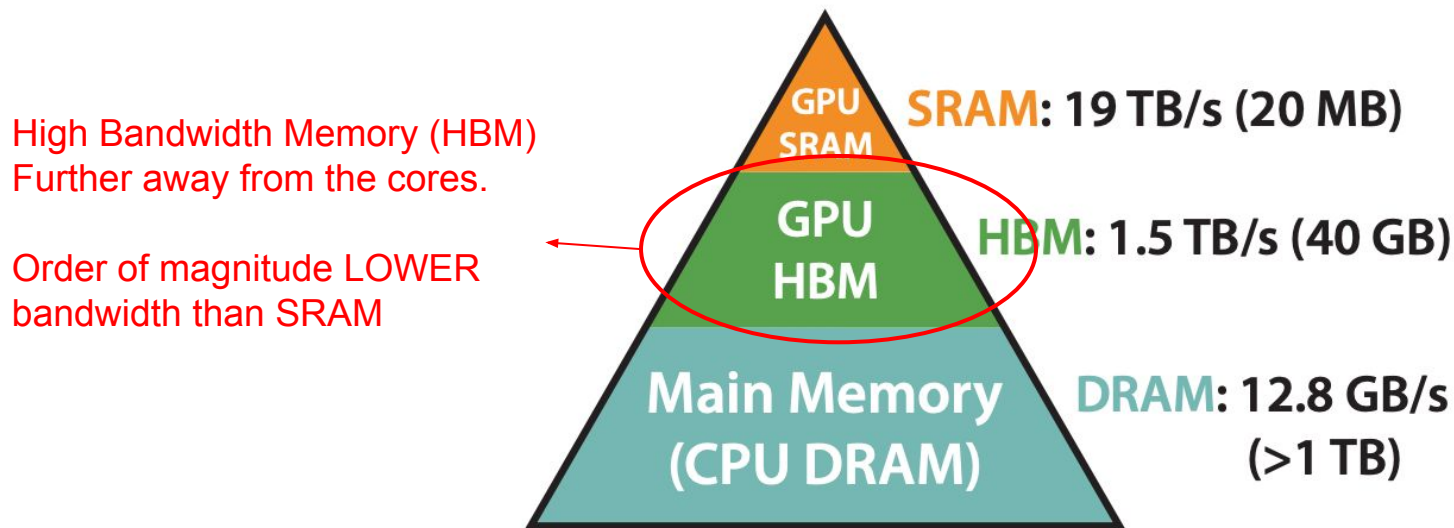
Memory hierarchy

20MB $\Rightarrow$ “only” $\sim 5 \times 10^6$ float32s.



Memory Hierarchy with
Bandwidth & Memory Size

Memory hierarchy



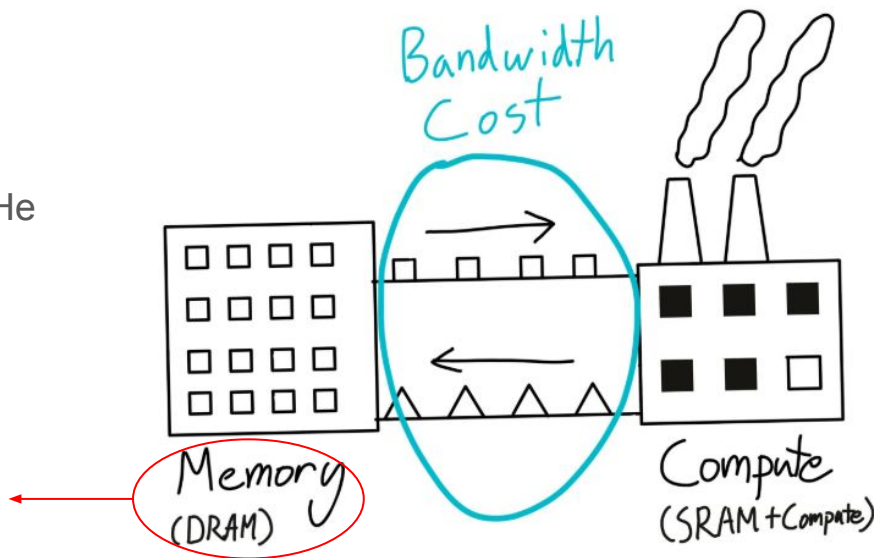
**Memory Hierarchy with
Bandwidth & Memory Size**

Another way of looking at it

1. All runtime is either compute or shuffling data

Image from a great
[blogpost](#) by Horace He

Equivalent to DRAM +
HBM in the
FlashAttention paper



Matmul as a primitive routine

1. In general, across all frameworks: Calls to a GEMM (General Matrix Multiply) routine usually results in the ***materialization*** of the output matrix
 - a. If the result can be stored in SRAM, store there, if not, write to HBM
 - b. Generally holds true across all frameworks

Matmul as a primitive routine

1. In general, across all frameworks: Calls to a GEMM (General Matrix Multiply) routine usually results in the *materialization* of the output matrix
 - a. If the result can be stored in SRAM, store there, if not, write to HBM
 - b. Generally holds true across all frameworks
2. As an example, look at what numpy does in [multiarraymodule.c](#)

Attention

<https://arxiv.org/abs/1706.03762>

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Attention

<https://arxiv.org/abs/1706.03762>

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

```
def single_headed_attention(q, k, v):  
    """Perform single-headed attention  
    Args:  
        q: (q_seq_len, dim)  
        k: (kv_seq_len, dim)  
        v: (kv_seq_len, dim)  
    Returns:  
        np.ndarray: (q_seq_len, dim)  
    """  
    scale = np.sqrt(k.shape[-1])  
    intermediate = (q@k.T) / scale  
    to_ret = softmax(intermediate) @ v  
    return to_ret
```

Algo 0

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Standard Attention: Doesn't matter how you write the code

```
def single_headed_attention(q, k, v):  
    """Perform single-headed attention  
    Args:  
        q: (q_seq_len, dim)  
        k: (kv_seq_len, dim)  
        v: (kv_seq_len, dim)  
    Returns:  
        np.ndarray: (q_seq_len, dim)  
    """  
  
    scale = np.sqrt(k.shape[-1])  
    intermediate = (q@k.T) / scale  
    to_ret = softmax(intermediate) @ v  
    return to_ret
```

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Standard Attention: Doesn't matter how you write the code

```
def single_headed_attention(q, k, v):  
    """Perform single-headed attention  
    Args:  
        q: (q_seq_len, dim)  
        k: (kv_seq_len, dim)  
        v: (kv_seq_len, dim)  
    Returns:  
        np.ndarray: (q_seq_len, dim)  
    """  
  
    scale = np.sqrt(k.shape[-1])  
    intermediate = (q@k.T) / scale  
    to_ret = softmax(intermediate) @ v  
    return to_ret
```

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Standard Attention: Doesn't matter how you write the code

```
def single_headed_attention(q, k, v):  
    """Perform single-headed attention  
  
    Args:  
        q: (q_seq_len, dim)  
        k: (kv_seq_len, dim)  
        v: (kv_seq_len, dim)  
  
    Returns:  
        np.ndarray: (q_seq_len, dim)  
    """  
    return softmax((q@k.T) / np.sqrt(k.shape[-1])) @ v
```

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Even if you rearrange how you write the operations to have no intermediate variables, the number of reads/writes to HBM remains unchanged - the primitive routine here is the matmul computation, and its results are always materialized in HBM!

Standard Attention: Doesn't matter how you write the code

```
def single_headed_attention(q, k, v):  
    """Perform single-headed attention  
    Args:  
        q: (q_seq_len, dim)  
        k: (kv_seq_len, dim)  
        v: (kv_seq_len, dim)  
    Returns:  
        np.ndarray: (q_seq_len, dim)  
    """  
    return softmax((q@k.T) / np.sqrt(k.shape[-1])) @ v
```

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Easier way to visualize

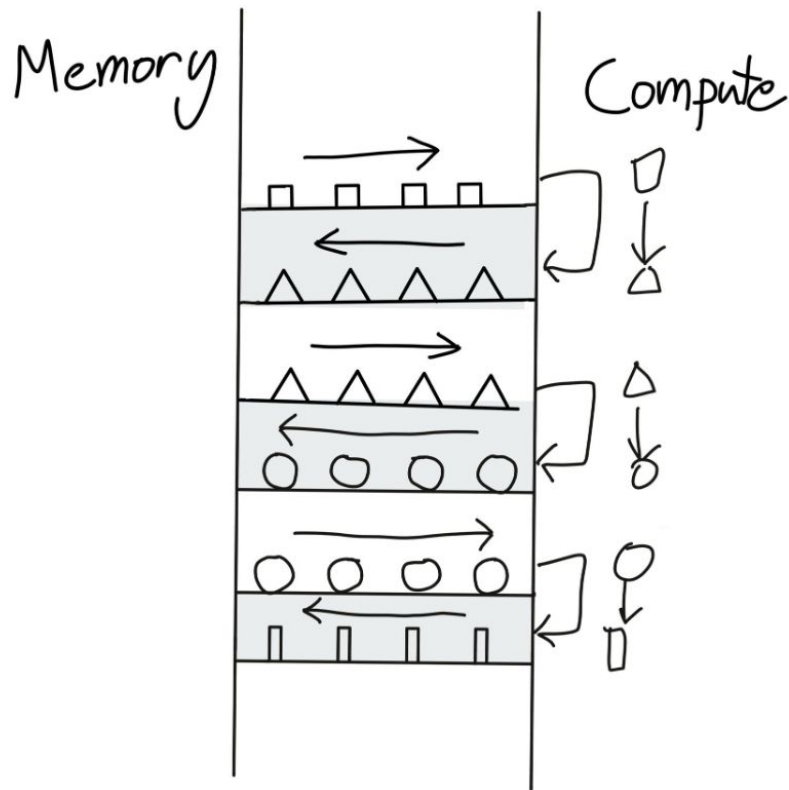
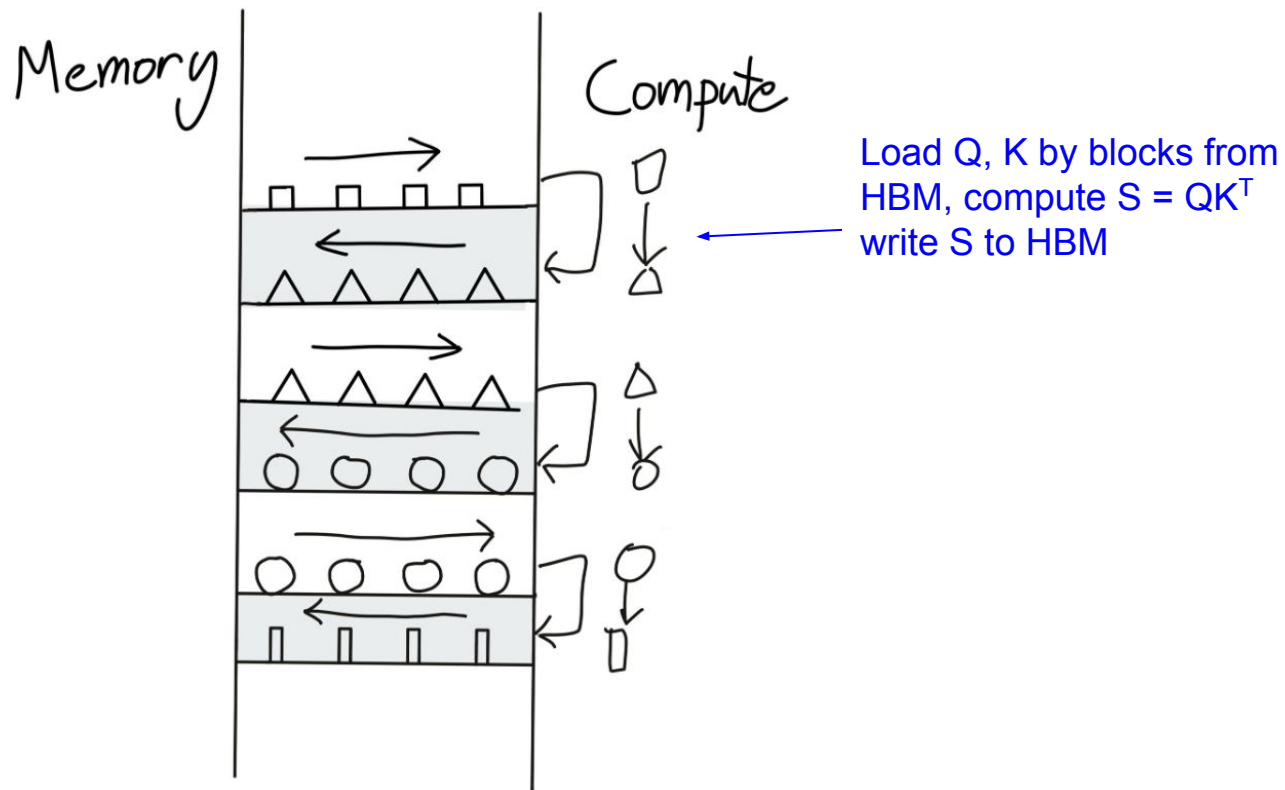
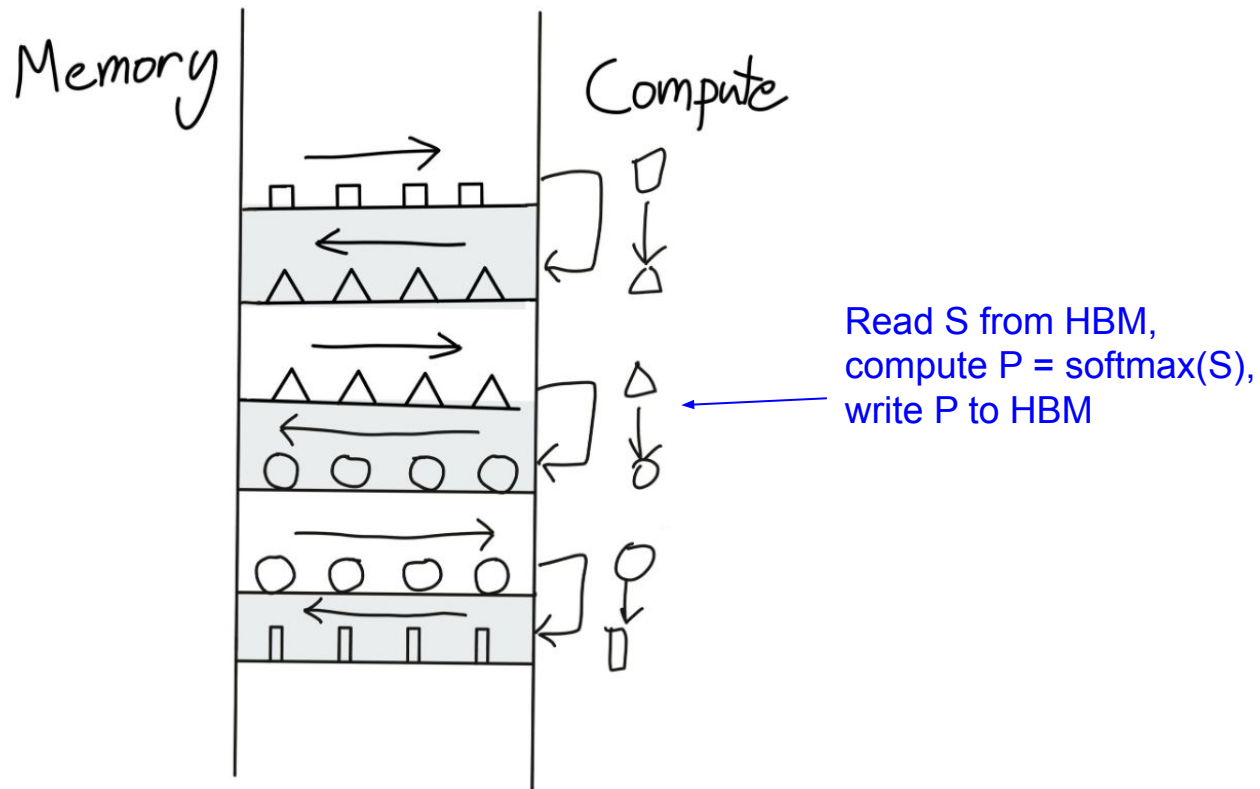


Image from the same great [blogpost](#) by Horace He

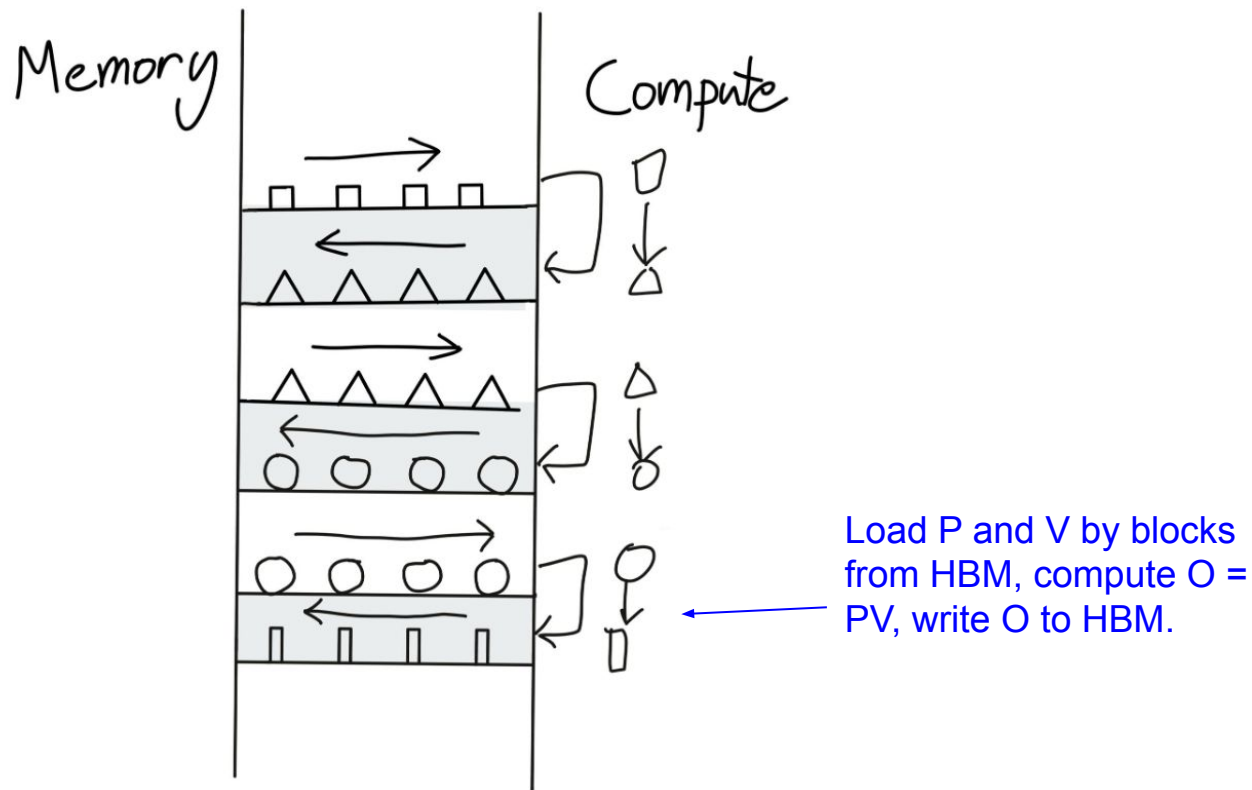
Easier way to visualize



Easier way to visualize



Easier way to visualize



Number of memory accesses

Theorem 2. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \leq M \leq Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while FLASHATTENTION (Algorithm 1) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.*

Number of memory accesses

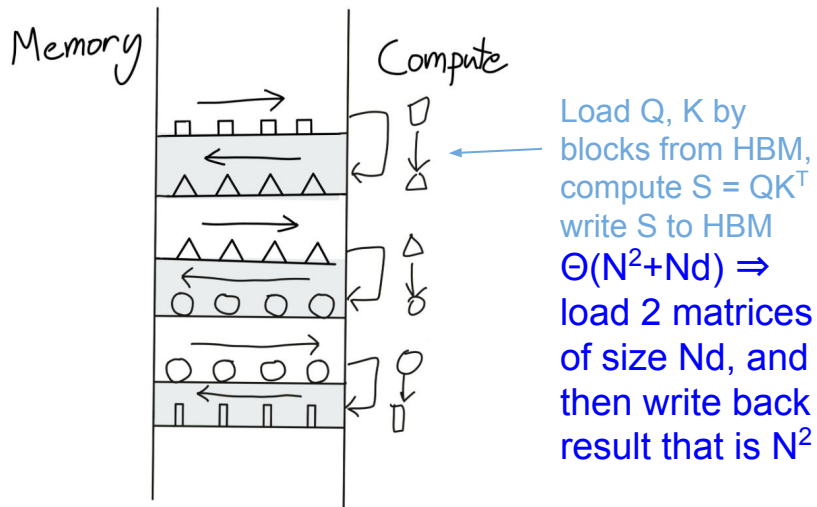
Theorem 2. Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \leq M \leq Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while FLASHATTENTION (Algorithm 1) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.

Big-theta notation (in case you have forgotten),
basically means “of the same order as”.
You **don't** get rid of the lower order Ns

$f(n) \asymp g(n)$ (Hardy's notation) or $f(n) = \Theta(g(n))$ (Knuth notation)	Of the same order as (Hardy); Big Theta (Knuth)	f is asymptotically bounded by g both above (with constant factor k_2) and below (with constant factor k_1)	$\exists k_1 > 0 \exists k_2 > 0 \exists n_0 \forall n > n_0:$ $k_1 g(n) \leq f(n) \leq k_2 g(n)$	$f(n) = O(g(n))$ and $g(n) = O(f(n))$
--	--	--	--	--

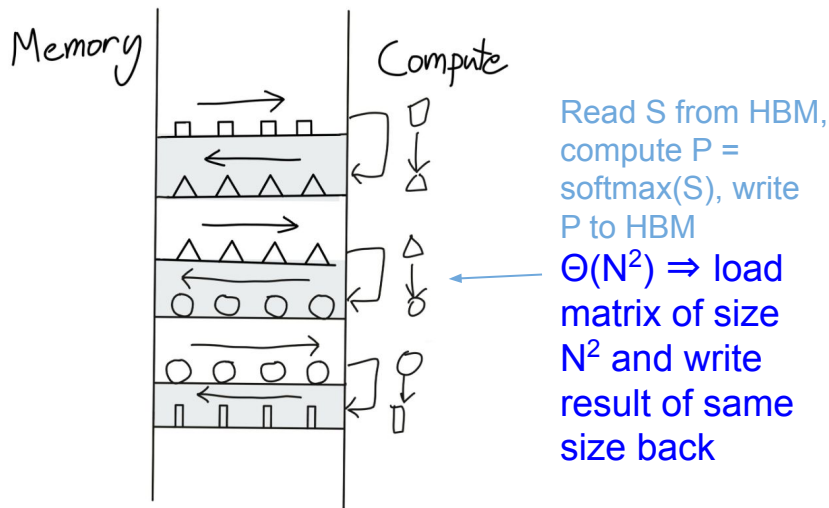
Number of memory accesses

Theorem 2. Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \leq M \leq Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while FLASHATTENTION (Algorithm 1) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.



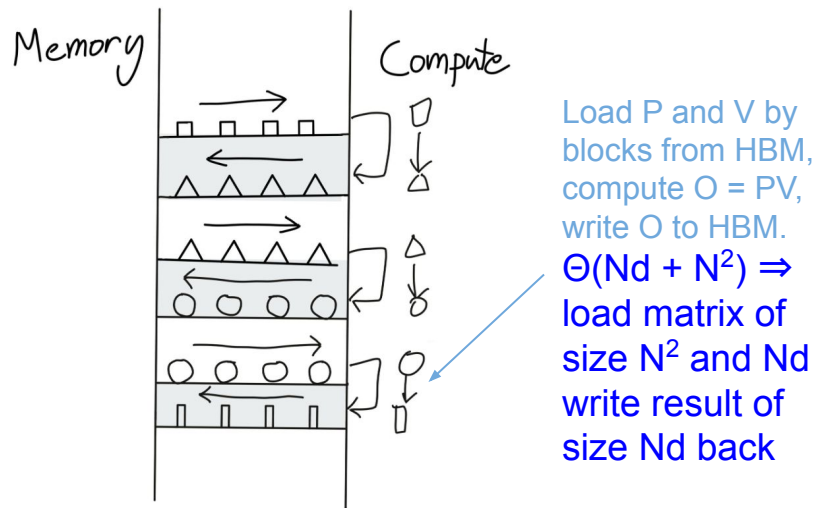
Number of memory accesses

Theorem 2. Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \leq M \leq Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while FLASHATTENTION (Algorithm 1) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.

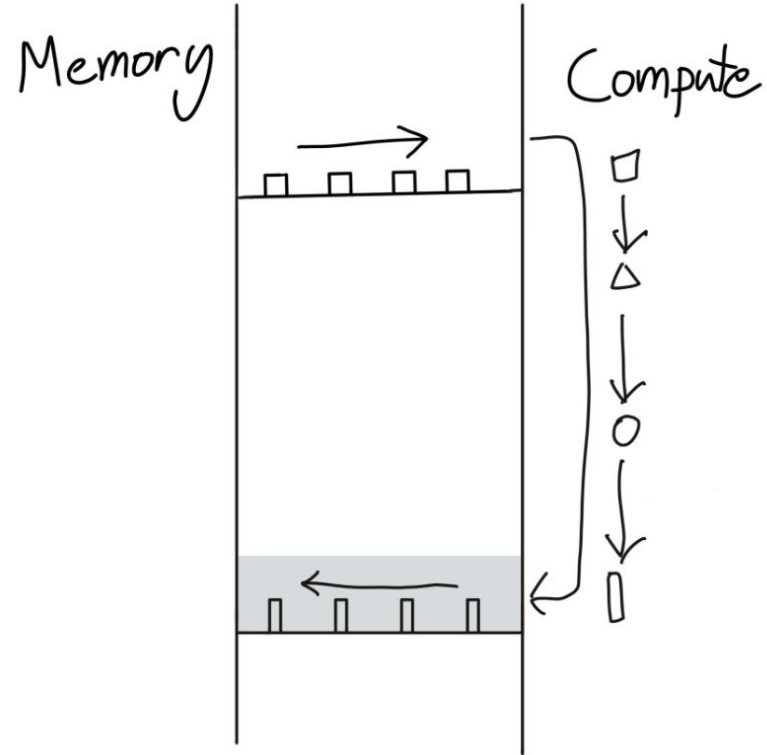


Number of memory accesses

Theorem 2. Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \leq M \leq Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while FLASHATTENTION (Algorithm 1) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.



What we want



Questions so far?

Outline

1. The basics
2. FlashAttention forward pass
3. FlashAttention backward pass
4. Brief notes on FlashAttention-2+3

FlashAttention - Forward + backward pass

Algorithm 2 FLASHATTENTION Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} .

- 1: Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
- 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 3: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 4: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 5: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 6: **for** $1 \leq j \leq T_c$ **do**
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: **for** $1 \leq i \leq T_r$ **do**
- 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 10: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 11: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
- 12: On chip, compute $\tilde{m}_{ij} = \text{rowsum}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 13: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 14: On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
- 15: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
- 16: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 17: **end for**
- 18: **end for**
- 19: **Return** $\mathbf{O}, \ell, m, \mathcal{R}$.

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} , pseudo-random number generator state $\mathcal{R}$ from the forward pass.

- 1: Set the pseudo-random number generator state to $\mathcal{R}$.
- 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}$, $\mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
- 6: **for** $1 \leq j \leq T_c$ **do**
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}$, $\mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
- 9: **for** $1 \leq i \leq T_r$ **do**
- 10: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 11: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 12: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
- 13: On chip, compute $\mathbf{P}_{ij} = \text{diag}(\ell_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
- 14: On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop} .
- 15: On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
- 16: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_{ij}^{\text{dropped}})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
- 17: On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 18: On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
- 19: On chip, compute $\mathbf{D}_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
- 20: On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - \mathbf{D}_i) \in \mathbb{R}^{B_r \times B_c}$.
- 21: Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
- 22: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \tau \mathbf{dS}_{ij}^T \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
- 23: **end for**
- 24: Write $\mathbf{dK}_j \leftarrow \mathbf{dK}_j$, $\mathbf{dV}_j \leftarrow \mathbf{dV}_j$ to HBM.
- 25: **end for**
- 26: **Return** $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.



Building intuition from scratch

1. Online softmax
2. Block matrix multiplication

“Standard” softmax

1. How do you implement softmax?

“Standard” softmax

1. How do you implement softmax?

- a. Reminder of definition: Given a vector x of N elements, the softmax is defined as -

$$s_i = \frac{e^{x_i}}{\sum_{j=0}^N e^{x_j}}$$

Where s_i is the output in the i th position

“Standard” softmax

1. How do you implement softmax?

- a. If the values of the vector are large, exponentials explode, and you may not be able to represent it in float32/float16.
- b. Standard trick - “scale” with $x_{\max}$, where $x_{\max}$ is the maximum value in the vector

$$s_i = \frac{e^{x_i - x_{\max}}}{\sum_{j=0}^N e^{x_j - x_{\max}}}$$

“Standard” softmax

1. How do you implement softmax?

```
Require  $\mathbf{X} \in \mathbb{R}^N$   
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m \leftarrow -\infty$   
for  $1 \leq i \leq N$  do:  
     $m \leftarrow \max(m, x_i)$   
end for  
 $l \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $l \leftarrow l + e^{x_i - m}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s \leftarrow \frac{e^{x_i - m}}{l}$   
end for
```

“Standard” softmax

1. How do you implement softmax?

```
Require  $\mathbf{X} \in \mathbb{R}^N$   
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m \leftarrow -\infty$   
for  $1 \leq i \leq N$  do:  
     $m \leftarrow \max(m, x_i)$   
end for  
 $l \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $l \leftarrow l + e^{x_i - m}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s \leftarrow \frac{e^{x_i - m}}{l}$   
end for
```

Need 3 reads
from memory
per element :(

“Online” softmax

1. How do you compute softmax with fewer reads from memory of each element?

```
Require  $\mathbf{X} \in \mathbb{R}^N$   
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m \leftarrow -\infty$   
for  $1 \leq i \leq N$  do:  
     $m \leftarrow \max(m, x_i)$   
end for  
 $l \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $l \leftarrow l + e^{x_i - m}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s \leftarrow \frac{e^{x_i - m}}{l}$   
end for
```

How to reduce
the number of for
loops where you
need the x_i ?

“Online” softmax

1. Intuition - fuse calculation of “max” element with the accumulation of exponents

“Online” softmax

1. Intuition - fuse calculation of “max” element with the accumulation of exponents

```
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m_0 \leftarrow -\infty$   
 $l_0 \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $m_i \leftarrow \max(m_{i-1}, x_i)$   
     $l_i \leftarrow l_{i-1} \times e^{m_{i-1}-m_i} + e^{x_i-m_i}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s_i \leftarrow \frac{e^{x_i-m_N}}{l_N}$   
end for
```

“Online” softmax

1. Intuition - fuse calculation of “max” element with the accumulation of exponents

```
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m_0 \leftarrow -\infty$   
 $l_0 \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $m_i \leftarrow \max(m_{i-1}, x_i)$   
     $l_i \leftarrow l_{i-1} \times e^{m_{i-1}-m_i} + e^{x_i-m_i}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s_i \leftarrow \frac{e^{x_i-m_N}}{l_N}$   
end for
```

Only 2 reads
per element -
what is the
trick?

“Online” softmax

1. Intuition - fuse calculation of “max” element with the accumulation of exponents

```
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m_0 \leftarrow -\infty$   
 $l_0 \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $m_i \leftarrow \max(m_{i-1}, x_i)$   
     $l_i \leftarrow l_{i-1} \times e^{m_{i-1}-m_i} + e^{x_i-m_i}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s_i \leftarrow \frac{e^{x_i-m_N}}{l_N}$   
end for
```

Apply a correction
term to l_{i-1} and
everything works

“Online” softmax

1. Is this correct? Yes! Proof by induction

```
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m_0 \leftarrow -\infty$   
 $l_0 \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $m_i \leftarrow \max(m_{i-1}, x_i)$   
     $l_i \leftarrow l_{i-1} \times e^{m_{i-1}-m_i} + e^{x_i-m_i}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s_i \leftarrow \frac{e^{x_i-m_N}}{l_N}$   
end for
```

Want to prove
that by the end
of this loop,

$$m_N = \max(x_i) = x_{\max}$$
$$l_N = \sum_{j=1}^N e^{x_j - x_{\max}}$$

"Online" softmax

Image from the same
[Github repo](#) of a pretty
cool guy on Youtube

1) Prove that it holds for a vector of size
 $N=1$

$$m_1 = \max(-\infty, x_1) = x_1 = \max_i(x_i) = x_{\max}$$

$$l_1 = 0 \times e^{-\infty} + e^{x_1 - x_1} = \sum_{j=1}^N e^{x_j - x_{\max}}$$

2) If we assume it holds for a vector of size
 N , does it hold for a vector of size $N+1$?

$$m_{N+1} = \max(m_N, x_{N+1}) = \max_i(x_i)$$

$$l_{N+1} = l_N e^{m_N - m_{N+1}} + e^{x_{N+1} - m_{N+1}} =$$
$$= \left(\sum_{j=1}^N e^{x_j - m_N} \right) e^{m_N - m_{N+1}} + e^{x_{N+1} - m_{N+1}}$$

$$= \sum_{j=1}^N e^{x_j - m_{N+1}} + e^{x_{N+1} - m_{N+1}}$$

$$= \sum_{j=1}^{N+1} e^{x_j - m_{N+1}}$$

Want to prove
that by the end
of this loop,

$$m_N = \max_i(x_i) = x_{\max}$$
$$l_N = \sum_{j=1}^N e^{x_j - x_{\max}}$$

Softmax in attention

1. How do you implement softmax?
 - a. For matrices:

Image from the [Github repo](#) of a pretty cool guy on Youtube

$$Q \in \mathbb{R}^{N \times d}, K \in \mathbb{R}^{N \times d}$$

$$S = QK^T \in \mathbb{R}^{N \times N}$$

$$P = \text{softmax}(S) \in \mathbb{R}^{N \times N}$$

$q_1^T k_1$	$q_1^T k_2$	$q_1^T k_3$	$q_1^T k_4$	$q_1^T k_5$
.	.			.
.		.		.
.			.	.
$q_5^T k_1$	$q_5^T k_2$	$q_5^T k_3$	$q_5^T k_4$	$q_5^T k_5$

(5,5)

SOFTMAX
→

0.1	0.05	0.5	0.15	0.2	$\Rightarrow \Sigma = 1$
.	.			.	
.		.		.	
.			.	.	
0.3	0.1	0.35	0.2	0.05	$\Rightarrow \Sigma = 1$

(5,5)

Softmax in attention

1. How do you implement softmax?
 - a. For matrices:

$$Q \in \mathbb{R}^{N \times d}, K \in \mathbb{R}^{N \times d}$$

$$S = QK^T \in \mathbb{R}^{N \times N}$$

$$P = \text{softmax}(S) \in \mathbb{R}^{N \times N}$$

```
def softmax(x):  
    """  
    Args:  
        x: (*, hidden_dim)  
    Returns:  
        np.ndarray: (*, hidden_dim)  
    """  
    exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))  
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

“Standard” softmax

1. How do you implement softmax?

```
def softmax(x):  
    """  
    Args:  
    | x: (*, hidden_dim)  
    Returns:  
    | np.ndarray: (*, hidden_dim) 2  
    """  
    exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True)) 1  
    return 3 exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

Require $\mathbf{X} \in \mathbb{R}^N$
Initialize $\mathbf{S} = (0)_N \in \mathbb{R}^N$
 $m \leftarrow -\infty$
for $1 \leq i \leq N$ do:
 $m \leftarrow \max(m, x_i)$
end for
 $l \leftarrow 0$
for $1 \leq i \leq N$ do:
 $l \leftarrow l + e^{x_i - m}$
end for
for $1 \leq i \leq N$ do:
 $s \leftarrow \frac{e^{x_i - m}}{l}$
end for

Need 3 reads
from memory
per element :(

Building intuition from scratch

1. Online softmax
2. Block matrix multiplication

Block matrix multiplication

1. Recap - what does block matrix multiplication mean?

- “Just” normal matrix multiplication, with partitioning

$$\begin{matrix} \boxed{A} & \times & \boxed{B} & = & \boxed{C} \\ (M, K) & & (K, N) & & (M, N) \end{matrix}$$

Split matrices into blocks of size $b_r \times b_c$.

In this example,

$$b_r = 4$$

$$b_c = 2$$

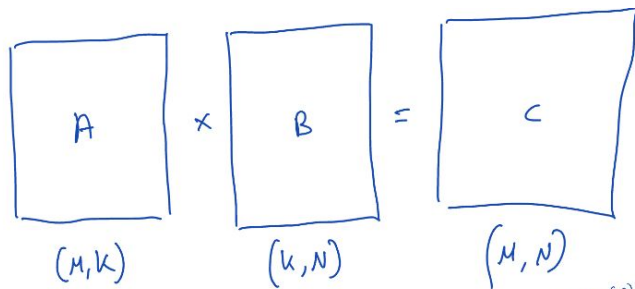
$$\begin{matrix} \begin{matrix} \boxed{A_{11}} & \boxed{A_{12}} \\ \boxed{A_{21}} & \boxed{A_{22}} \end{matrix} & \times & \begin{matrix} \boxed{B_{11}} & \boxed{B_{12}} \\ \boxed{B_{21}} & \boxed{B_{22}} \end{matrix} & = & \begin{matrix} \boxed{A_{11}B_{11} + A_{12}B_{21}} & \boxed{A_{11}B_{12} + A_{12}B_{22}} \\ \boxed{A_{21}B_{11} + A_{22}B_{21}} & \boxed{A_{21}B_{12} + A_{22}B_{22}} \end{matrix} \\ \text{ORIGINAL}(8, 4) & & \text{ORIGINAL}(4, 8) & & \text{ORIGINAL}(8, 8) \\ \text{BLOCK}(2, 2) & & \text{BLOCK}(2, 2) & & \text{BLOCK}(2, 2) \end{matrix}$$

Illustrations from the same [Github repo](#) of a pretty cool guy on Youtube

Block matrix multiplication

1. Recap - what does block matrix multiplication mean?

- “Just” normal matrix multiplication, with partitioning

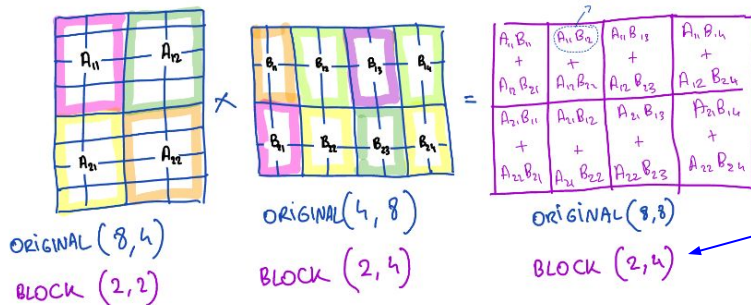


Split matrices into blocks of size $b_r \times b_c$.

In this example,

$$b_r = 4$$

$$b_c = 2$$



From a “block” perspective, this shape is $(M/b_r, N/b_c)$

Block matrix multiplication

1. Can't partition anyhow...

Multiplication [\[edit\]](#)

It is possible to use a block partitioned matrix product that involves only algebra on submatrices of the factors. The partitioning of the factors is not arbitrary, however, and requires "conformable partitions"^[12] between two matrices A and B such that all submatrix products that will be used are defined.^[13]

Two matrices A and B are said to be partitioned conformally for the product AB , when A and B are partitioned into submatrices and if the multiplication AB is carried out treating the submatrices as if they are scalars, but keeping the order, and when all products and sums of submatrices involved are defined.

— Arak M. Mathai and Hans J. Haubold, *Linear Algebra: A Course for Physicists and Engineers*^[14]

... but this is kind of a detail for now!

Block matrix multiplication

1. How would you compute the following quantity?
 - a. Basically output of attention without the softmax

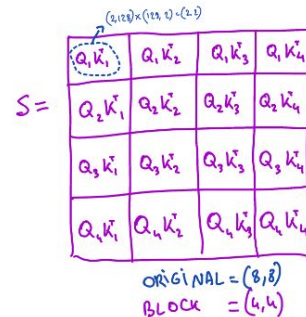
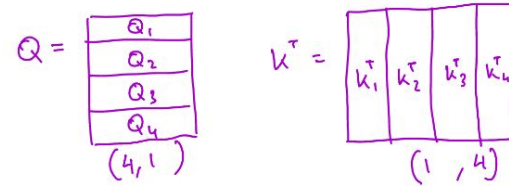
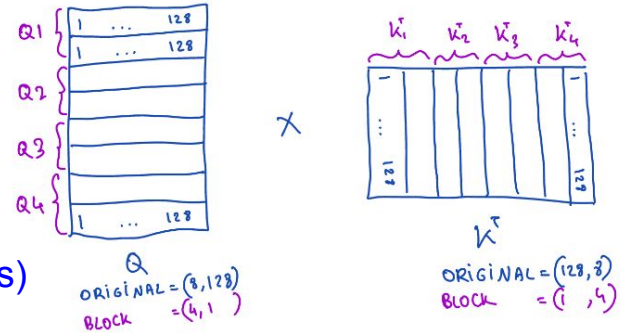
$$O = (QK^T)V \in \mathbb{R}^{N \times d}$$

Block matrix multiplication

1. Do QK^T first

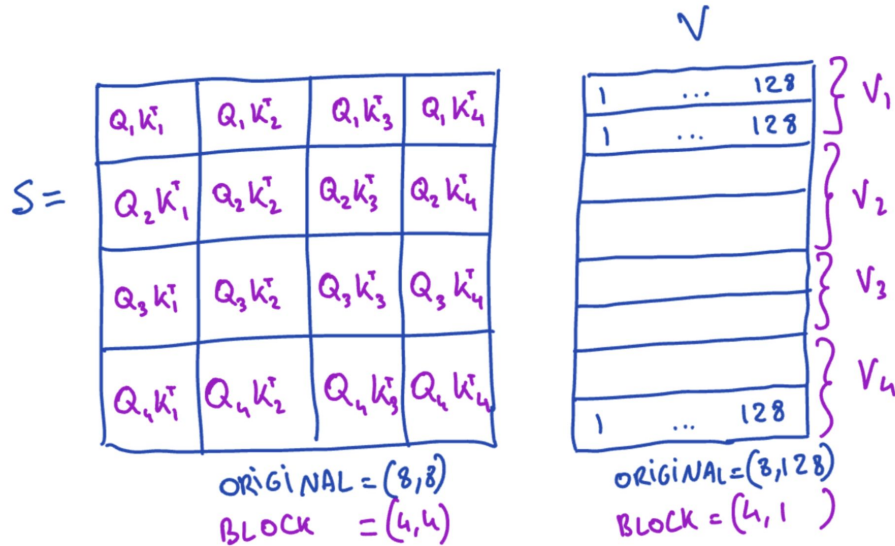
Split matrices into blocks of size $b \times d$

In this example,
 $b = 2$,
 $d = 128$, and
 n (number of rows)
 $= 8$

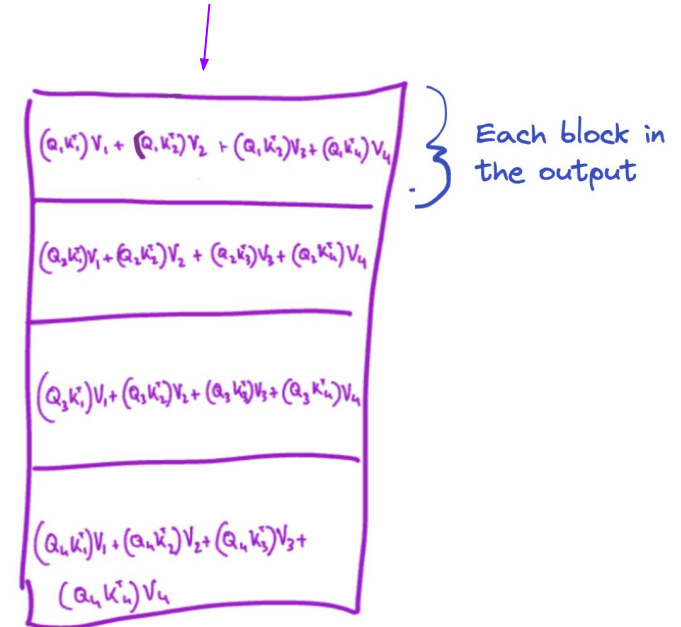


Fused block matrix multiplication

1. Then multiply by V



Final result



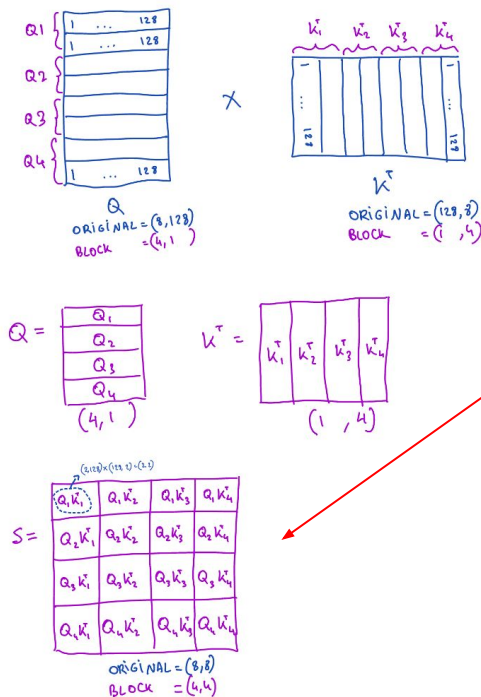
“Unfused” way

```
Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{S} = (0)_{N \times N} \in \mathbb{R}^{N \times N}$ ,  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lfloor \sqrt{M} \rfloor$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{S}$  into  $T_s = \frac{N^2}{B^2}$  blocks
 $\mathbf{S}_{11}, \mathbf{S}_{12}, \dots, \mathbf{S}_{1, \frac{N}{B}}$ ,
 $\mathbf{S}_{21}, \mathbf{S}_{22}, \dots, \mathbf{S}_{2, \frac{N}{B}}$ ,
...  $\mathbf{S}_{\frac{N}{B}, \frac{N}{B}}$ ,
for  $1 \leq i \leq T$  do
    Load  $\mathbf{Q}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{S}_{ij}$  from HBM to on-chip SRAM
         $\mathbf{S}_{ij} \leftarrow \mathbf{Q}_i \mathbf{K}_j^T$ 
        Write  $\mathbf{S}_{ij}$  to HBM
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{V}_j, \mathbf{S}_{ij}$  from HBM to on-chip SRAM
         $\mathbf{O}_i \leftarrow \mathbf{O}_i + \mathbf{S}_{ij} \mathbf{V}_j$ 
    write  $\mathbf{O}_i$  to HBM
return  $\mathbf{O}$ 
```

“Unfused” way

Do QK^T first

In this example,
 $b = 2$,
 $d = 128$, and
 n (number of rows) $= 8$



Require matrices $Q, K, V \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M
 Initialize $S = (0)_{N \times N} \in \mathbb{R}^{N \times N}$, $O = (0)_{N \times d} \in \mathbb{R}^{N \times d}$
 Set block size $B = \lfloor \sqrt{M} \rfloor$
 Split Q, K, V, O into $T = \lceil \frac{N}{B} \rceil$ blocks

**$Q_1, Q_2, \dots, Q_T$,
 $K_1, K_2, \dots, K_T$,
 $V_1, V_2, \dots, V_T$,
 $O_1, O_2, \dots, O_T$**
 each of size $\mathbb{R}^{B \times d}$

Split S into $T_s = \frac{N^2}{B^2}$ blocks

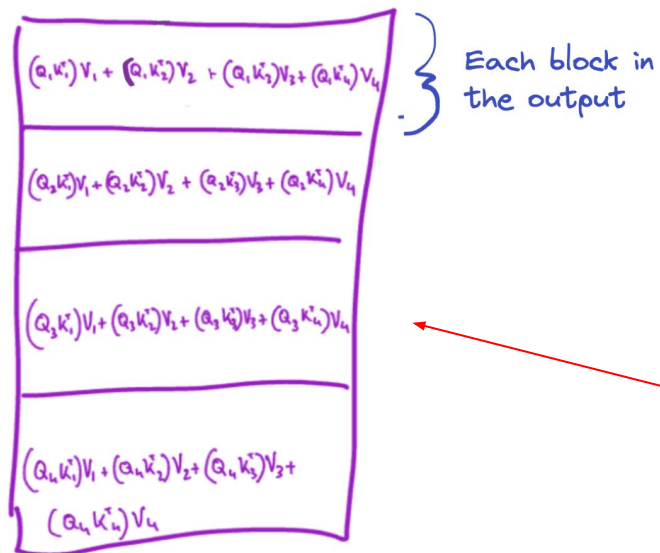
$S_{11}, S_{12}, \dots, S_{1, \frac{N}{B}}$,
 $S_{21}, S_{22}, \dots, S_{2, \frac{N}{B}}$,
 $\dots, S_{\frac{N}{B}, \frac{N}{B}}$,

for $1 \leq i \leq T$ do:
 Load Q_i from HBM to on-chip SRAM
 for $1 \leq j \leq T$ do:
 Load K_j, S_{ij} from HBM to on-chip SRAM
 $S_{ij} \leftarrow Q_i K_j^T$
 Write S_{ij} to HBM

for $1 \leq i \leq T$ do:
 Load O_i from HBM to on-chip SRAM
 for $1 \leq j \leq T$ do:
 Load V_j, S_{ij} from HBM to on-chip SRAM
 $O_i \leftarrow O_i + S_{ij} V_j$
 write O_i to HBM

return O

“Unfused” way



Each block in the output

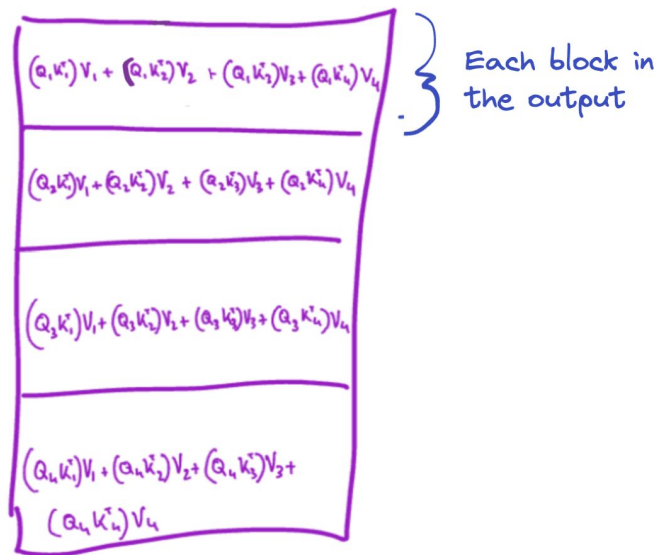
$$\begin{aligned} & (Q_1 K_1^T) V_1 + (Q_1 K_2^T) V_2 + (Q_1 K_3^T) V_3 + (Q_1 K_4^T) V_4 \\ & (Q_2 K_1^T) V_1 + (Q_2 K_2^T) V_2 + (Q_2 K_3^T) V_3 + (Q_2 K_4^T) V_4 \\ & (Q_3 K_1^T) V_1 + (Q_3 K_2^T) V_2 + (Q_3 K_3^T) V_3 + (Q_3 K_4^T) V_4 \\ & (Q_4 K_1^T) V_1 + (Q_4 K_2^T) V_2 + (Q_4 K_3^T) V_3 + (Q_4 K_4^T) V_4 \end{aligned}$$

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{S} = (\mathbf{0})_{N \times N} \in \mathbb{R}^{N \times N}$ ,  $\mathbf{O} = (\mathbf{0})_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lfloor \sqrt{M} \rfloor$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{S}$  into  $T_s = \frac{N^2}{B^2}$  blocks
 $\mathbf{S}_{11}, \mathbf{S}_{12}, \dots, \mathbf{S}_{1, \frac{N}{B}}$ ,
 $\mathbf{S}_{21}, \mathbf{S}_{22}, \dots, \mathbf{S}_{2, \frac{N}{B}}$ ,
 $\dots, \mathbf{S}_{\frac{N}{B}, \frac{N}{B}}$ ,
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{S}_{ij}$  from HBM to on-chip SRAM
         $\mathbf{S}_{ij} \leftarrow \mathbf{Q}_i \mathbf{K}_j^T$ 
        Write  $\mathbf{S}_{ij}$  to HBM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{O}_i$  from HBM to on-chip SRAM
        for  $1 \leq j \leq T$  do:
            Load  $\mathbf{V}_j, \mathbf{S}_{ij}$  from HBM to on-chip SRAM
             $\mathbf{O}_i \leftarrow \mathbf{O}_i + \mathbf{S}_{ij} \mathbf{V}_j$ 
        write  $\mathbf{O}_i$  to HBM
return  $\mathbf{O}$ 
    
```

Fused block matrix multiplication

1. How would you do this with fewer read/writes to HBM?



Each block in the output

$$\begin{aligned} & (Q_1 K_1^T) V_1 + (Q_1 K_2^T) V_2 + (Q_1 K_3^T) V_3 + (Q_1 K_4^T) V_4 \\ & (Q_2 K_1^T) V_1 + (Q_2 K_2^T) V_2 + (Q_2 K_3^T) V_3 + (Q_2 K_4^T) V_4 \\ & (Q_3 K_1^T) V_1 + (Q_3 K_2^T) V_2 + (Q_3 K_3^T) V_3 + (Q_3 K_4^T) V_4 \\ & (Q_4 K_1^T) V_1 + (Q_4 K_2^T) V_2 + (Q_4 K_3^T) V_3 + (Q_4 K_4^T) V_4 \end{aligned}$$

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (\mathbf{0})_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T,$ 
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T,$ 
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T,$ 
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
         $\mathbf{O}_i \leftarrow \mathbf{O}_i + (\mathbf{Q}_i \mathbf{K}_j^T) \mathbf{V}_j$ 
    end for
    Write  $\mathbf{O}_i$  to HBM
end for
return  $\mathbf{O}$ 
    
```

Fused block matrix multiplication

$$\begin{array}{|l}
 (Q_1 K_1^T) V_1 + (Q_1 K_2^T) V_2 + (Q_1 K_3^T) V_3 + (Q_1 K_4^T) V_4 \\
 \hline
 (Q_2 K_1^T) V_1 + (Q_2 K_2^T) V_2 + (Q_2 K_3^T) V_3 + (Q_2 K_4^T) V_4 \\
 \hline
 (Q_3 K_1^T) V_1 + (Q_3 K_2^T) V_2 + (Q_3 K_3^T) V_3 + (Q_3 K_4^T) V_4 \\
 \hline
 (Q_4 K_1^T) V_1 + (Q_4 K_2^T) V_2 + (Q_4 K_3^T) V_3 + (Q_4 K_4^T) V_4
 \end{array}
 \left. \vphantom{\begin{array}{|l} \end{array}} \right\} \text{Each block in the output}$$

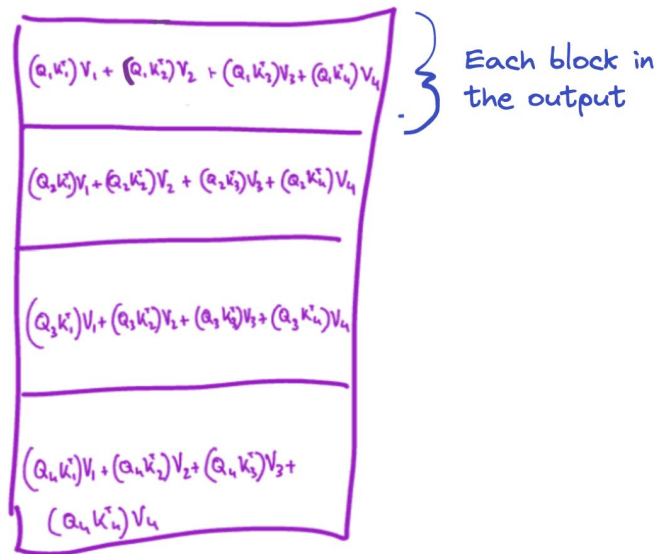
Why divide it by $4d$?

We have O_i, Q_i, K_j, V_j in SRAM at any one time, each of size $B \times d$

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (\mathbf{0})_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
         $\mathbf{O}_i \leftarrow \mathbf{O}_i + (\mathbf{Q}_i \mathbf{K}_j^T) \mathbf{V}_j$ 
    end for
    Write  $\mathbf{O}_i$  to HBM
end for
return  $\mathbf{O}$ 
    
```

Fused block matrix multiplication



Each block in the output

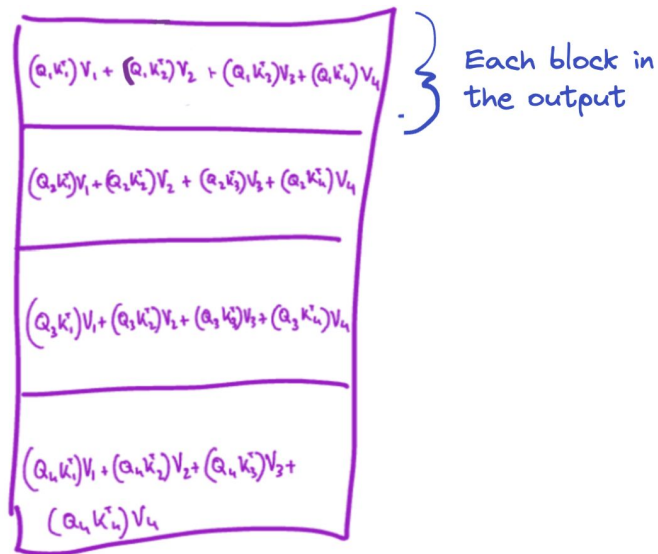
$$\begin{aligned} & (Q_1 K_1^T) V_1 + (Q_1 K_2^T) V_2 + (Q_1 K_3^T) V_3 + (Q_1 K_4^T) V_4 \\ & (Q_2 K_1^T) V_1 + (Q_2 K_2^T) V_2 + (Q_2 K_3^T) V_3 + (Q_2 K_4^T) V_4 \\ & (Q_3 K_1^T) V_1 + (Q_3 K_2^T) V_2 + (Q_3 K_3^T) V_3 + (Q_3 K_4^T) V_4 \\ & (Q_4 K_1^T) V_1 + (Q_4 K_2^T) V_2 + (Q_4 K_3^T) V_3 + (Q_4 K_4^T) V_4 \end{aligned}$$

Actually you'll need 2 different block sizes, but that will be addressed later

```
Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T,$ 
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T,$ 
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T,$ 
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
         $\mathbf{O}_i \leftarrow \mathbf{O}_i + (\mathbf{Q}_i \mathbf{K}_j^T) \mathbf{V}_j$ 
    end for
    Write  $\mathbf{O}_i$  to HBM
end for
return  $\mathbf{O}$ 
```

Fused block matrix multiplication

You save the N^2 writes to HBM of the intermediate S matrix!



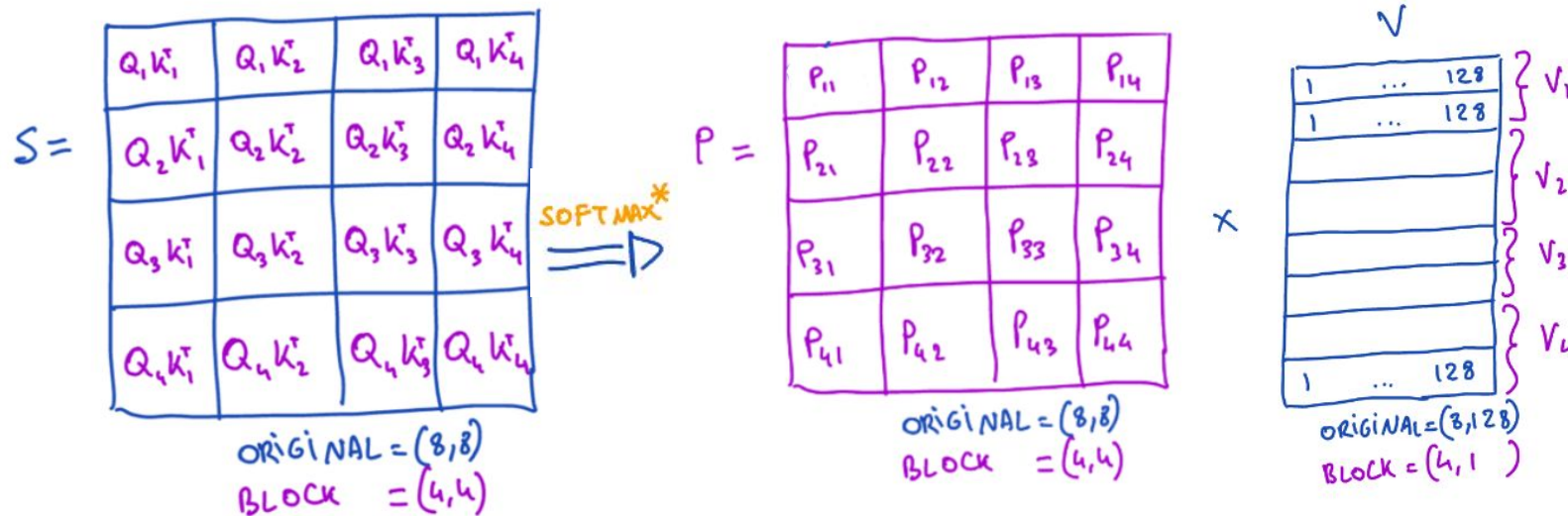
Each block in the output

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (\mathbf{0})_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T,$ 
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T,$ 
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T,$ 
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
         $\mathbf{O}_i \leftarrow \mathbf{O}_i + (\mathbf{Q}_i \mathbf{K}_j^T) \mathbf{V}_j$ 
    end for
    Write  $\mathbf{O}_i$  to HBM
end for
return  $\mathbf{O}$ 
    
```

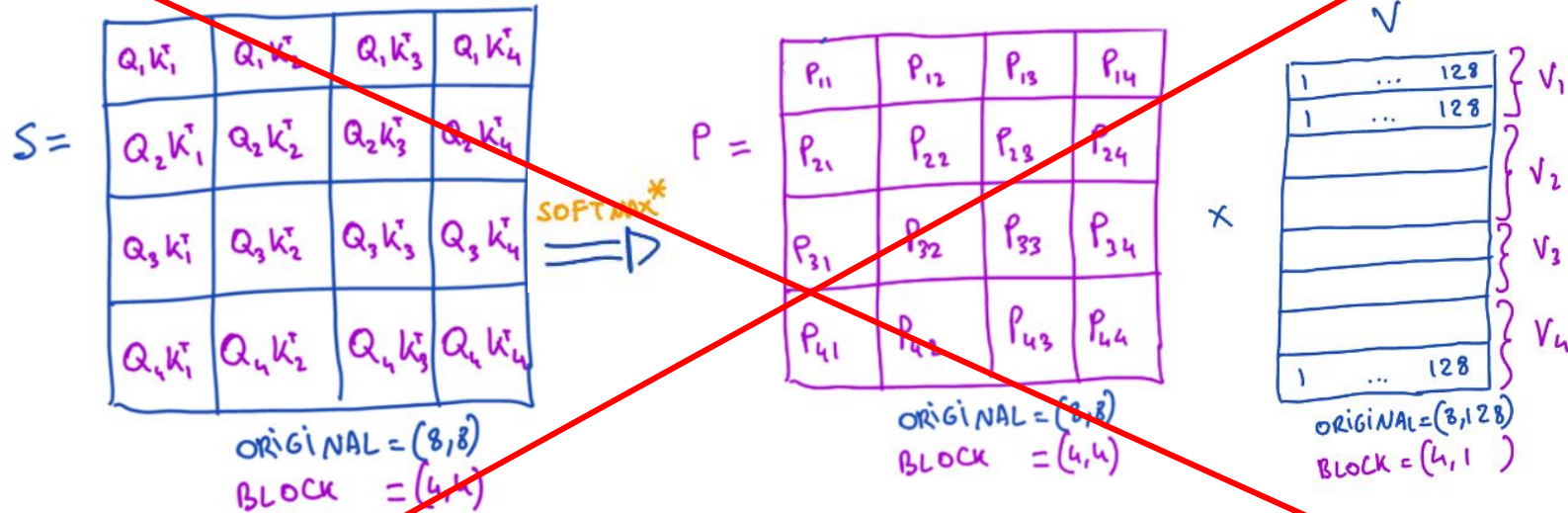
Now bring in softmax - Can you just do this?

Illustrations from the same [Github repo](#) of a pretty cool guy on Youtube



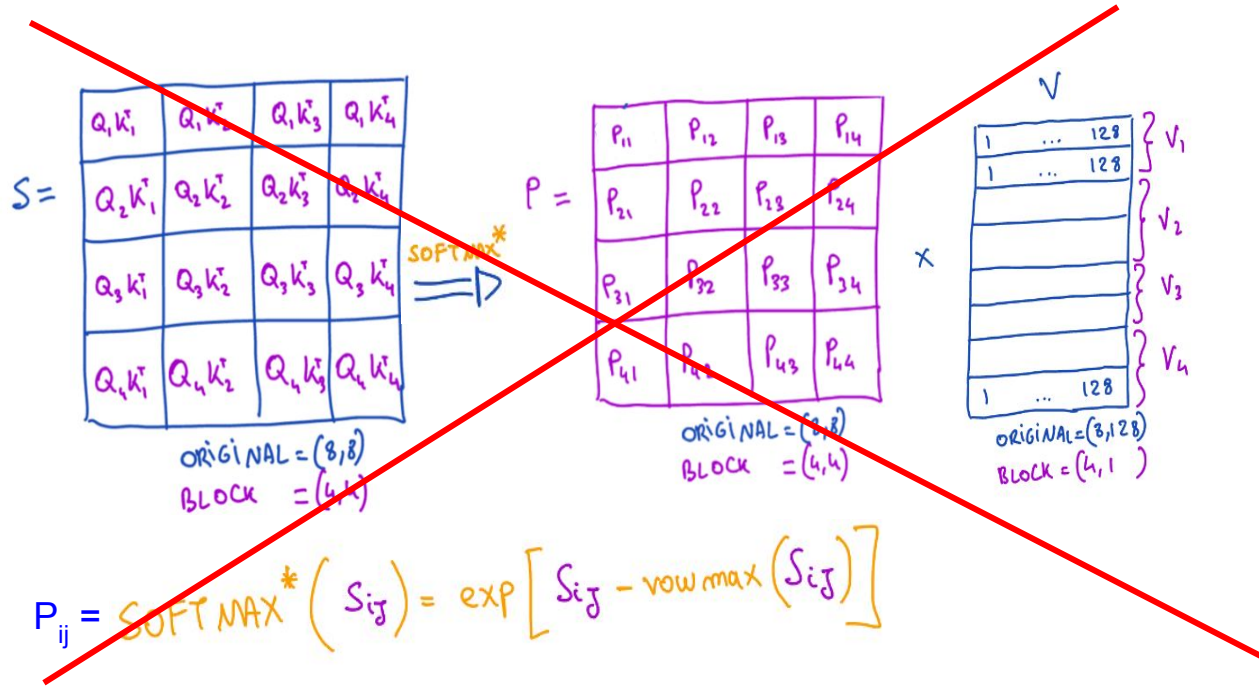
$$P_{ij} = \text{SOFTMAX}^* (S_{ij}) = \exp \left[S_{ij} - \text{row max} (S_{ij}) \right]$$

Now bring in softmax - Can you just do this?



$$\text{SOFTMAX}^*(S_{ij}) = \exp[S_{ij} - \text{row max}(S_{ij})]$$

Now bring in softmax - Can you just do this?



The rowmax for each P_{ij} block is *not* the global rowmax for each row!
Your result will be wrong!

Now bring in softmax - What does it entail?

1. Use the same “trick” 1D online softmax calculation, but this time use it for 2D matrices!

```
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$   
 $m_0 \leftarrow -\infty$   
 $l_0 \leftarrow 0$   
for  $1 \leq i \leq N$  do:  
     $m_i \leftarrow \max(m_{i-1}, x_i)$   
     $l_i \leftarrow l_{i-1} \times e^{m_{i-1}-m_i} + e^{x_i-m_i}$   
end for  
for  $1 \leq i \leq N$  do:  
     $s_i \leftarrow \frac{e^{x_i-m_N}}{l_N}$   
end for
```

Now bring in softmax - What does it entail?

1. Use the same “trick” 1D online softmax calculation, but this time use it for 2D matrices! $\Rightarrow$ Fuse operations in SRAM

```
Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ 
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
         $\mathbf{O}_i \leftarrow \mathbf{O}_i + (\mathbf{Q}_i \mathbf{K}_j^T) \mathbf{V}_j$ 
    end for
    Write  $\mathbf{O}_i$  to HBM
end for
return  $\mathbf{O}$ 
```

Previous block matrix
pseudocode + 1D online
softmax
Do more “stuff” here
when data is loaded into
SRAM!

```
Initialize  $\mathbf{S} = (0)_N \in \mathbb{R}^N$ 
 $m_0 \leftarrow -\infty$ 
 $l_0 \leftarrow 0$ 
for  $1 \leq i \leq N$  do:
     $m_i \leftarrow \max(m_{i-1}, x_i)$ 
     $l_i \leftarrow l_{i-1} \times e^{m_{i-1} - m_i} + e^{x_i - m_i}$ 
end for
for  $1 \leq i \leq N$  do:
     $s_i \leftarrow \frac{e^{x_i - m_N}}{l_N}$ 
end for
```

Now bring in softmax - What does it entail?

```
Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $l = (0)_N \in \mathbb{R}^N$ ,  $m = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $l$  and  $m$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, l_i, m_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{l}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $l_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} l_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{l}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(l_i^{\text{new}})^{-1} (\text{diag}(l_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
        Write  $l_i \leftarrow l_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 
```

Now bring in softmax - What does it entail?

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{l} = (0)_N \in \mathbb{R}^N$ ,  $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{l}$  and  $\mathbf{m}$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
        Write  $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}, \mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 

```

Outer loop iterates “by column”, vs usual block matrix pseudocode

```

for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i$  from HBM to on-chip SRAM
    for  $1 \leq j \leq T$  do:
        Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM

```

Now bring in softmax - What does it entail?

```
Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{l} = (0)_N \in \mathbb{R}^N$ ,  $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{l}$  and  $\mathbf{m}$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
  Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
  for  $1 \leq i \leq T$  do:
    Load  $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$  from HBM to on-chip SRAM
    On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
    On chip, compute  $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
    On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
    On chip, compute  $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
    On chip, compute  $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^B$ 
    On chip, compute  $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^B$ 
    Write  $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
    Write  $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}, \mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$  to HBM
  end for
end for
return  $\mathbf{O}$ 
```

Aggregates for softmax computations are now vectors (instead of constants in the 1D case)

That's (basically) it for forward pass!

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\ell = (0)_N \in \mathbb{R}^N$ ,  $m = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\ell$  and  $m$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{\ell}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$  to HBM
        Write  $\ell_i \leftarrow \ell_i^{\text{new}}$ ,  $m_i \leftarrow m_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 

```

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

“Intuitive” build up from scratch

Forward pass of FlashAttention (without dropout + masking)

That's (basically) it for forward pass!

In reality, you need 2 different block sizes,
with B_r “capped” at size d

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{O}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

Forward pass of FlashAttention (without dropout + masking)

That's (basically) it for forward pass!

Need the following conditions met:

$$2B_r d + 2B_c d \leq M, \text{ and}$$

$$B_r B_c \leq M$$

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{O}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: **Return** $\mathbf{O}$.
-

Forward pass of FlashAttention (without dropout + masking)

That's (basically) it for forward pass!

Algorithm 2 FLASHATTENTION Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} .

- 1: Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
- 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 3: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 4: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 5: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 6: **for** $1 \leq j \leq T_c$ **do**
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: **for** $1 \leq i \leq T_r$ **do**
- 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 10: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 11: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
- 12: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 13: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 14: On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
- 15: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
- 16: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 17: **end for**
- 18: **end for**
- 19: Return $\mathbf{O}, \ell, m, \mathcal{R}$.

Forward pass of FlashAttention (**with** dropout + masking)

That's (basically) it for forward pass!

Algorithm 2 FLASHATTENTION Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} .

- 1: Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
 - 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 3: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 4: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 5: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: **for** $1 \leq i \leq T_r$ **do**
 - 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 10: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 11: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
 - 12: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 13: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 14: On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
 - 15: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
 - 16: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 17: **end for**
 - 18: **end for**
 - 19: Return $\mathbf{O}, \ell, m, \mathcal{R}$.
-

Trivially add
masking and
dropout

Forward pass of FlashAttention (**with** dropout + masking)

Read/writes to/from HBM

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{l} = (0)_N \in \mathbb{R}^N$ ,  $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{l}$  and  $\mathbf{m}$  into  $T$  blocks as well, each of size  $B$ 
for 1  $\leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for 1  $\leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
        Write  $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}, \mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 

```

Each $\mathbf{K}_j, \mathbf{V}_j$ loaded once from HBM.
 $\Theta(Nd)$ reads from HBM

Read/writes to/from HBM

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{l} = (0)_N \in \mathbb{R}^N$ ,  $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{l}$  and  $\mathbf{m}$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
        Write  $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}, \mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 

```

Each $\mathbf{Q}_i, \mathbf{O}_j$ loaded T times from HBM and written back to it
 $\Theta(NdT)$ accesses to HBM

Read/writes to/from HBM

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{l} = (0)_N \in \mathbb{R}^N$ ,  $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{l}$  and  $\mathbf{m}$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
        Write  $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}$ ,  $\mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 

```

Overall,
 $\Theta(NdT + Nd) = \Theta(NdT)$
 accesses to HBM

Substitute $T = 4Nd/M$
 $\Theta(NdT) = \Theta(N^2 d^2 M^{-1})$

Read/writes to/from HBM

```

Require matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$  in HBM, on-chip SRAM of size  $M$ 
Initialize  $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{l} = (0)_N \in \mathbb{R}^N$ ,  $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$  in HBM
Set block size  $B = \lceil \frac{M}{4d} \rceil$ 
Split  $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$  into  $T = \lceil \frac{N}{B} \rceil$  blocks
 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_T$ ,
 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_T$ ,
 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_T$ ,
 $\mathbf{O}_1, \mathbf{O}_2, \dots, \mathbf{O}_T$ 
each of size  $\mathbb{R}^{B \times d}$ 
Split  $\mathbf{l}$  and  $\mathbf{m}$  into  $T$  blocks as well, each of size  $B$ 
for  $1 \leq j \leq T$  do:
    Load  $\mathbf{K}_j, \mathbf{V}_j$  from HBM to on-chip SRAM
    for  $1 \leq i \leq T$  do:
        Load  $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$  from HBM to on-chip SRAM
        On chip, compute  $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B \times B}$ 
        On chip, compute  $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B \times B}$  (compute pointwise)
        On chip, compute  $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\mathbf{P}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^B$ 
        On chip, compute  $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^B$ 
        Write  $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \mathbf{P}_{ij} \mathbf{V}_j)$  to HBM
        Write  $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}, \mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$  to HBM
    end for
end for
return  $\mathbf{O}$ 

```

For standard values of d (64 - 128) and M (around 100kB), you'll get

$$\Theta(N^2 d^2 M^{-1}) \ll \Theta(Nd + N^2)$$

That's (basically) it for forward pass!

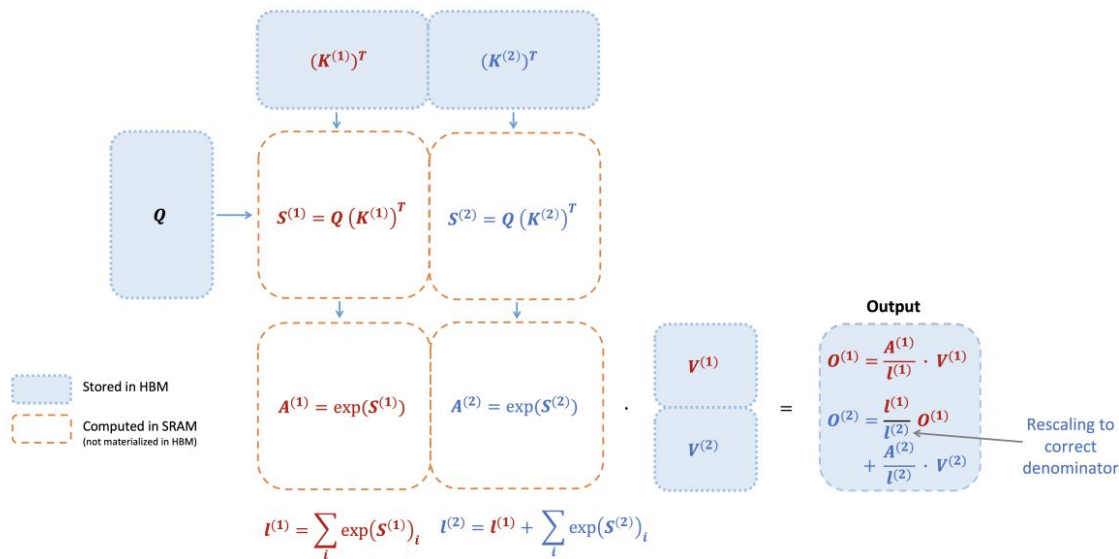
1. You now have a primitive routine just for attention! (after you take care of backward pass)

That's (basically) it for forward pass!

1. You now have a primitive routine just for attention! (after you take care of backward pass)
2. Can implement this in your framework of choice (CUDA/cuDNN/Triton) such that your Python code sees ***attention*** as the basic primitive instead of ***matmul***

That's (basically) it for forward pass!

1. Zooming out:



Questions so far?

Outline

1. The basics
2. FlashAttention forward pass
3. FlashAttention backward pass
4. Brief notes on FlashAttention-2+3

How about backward pass?

1. What is the “backward pass” of the attention mechanism?

How about backward pass?

1. What is the “backward pass” of the attention mechanism?
 - a. Let's stick with these variable names

$$\begin{array}{c} \begin{array}{c} \uparrow \\ \mathbb{R}^{N \times N} \end{array} S = \begin{array}{c} \mathbb{R}^{N \times d} \\ \downarrow \\ Q \end{array} \begin{array}{c} \mathbb{R}^{d \times N} \\ \leftarrow \\ K^T \end{array}, \quad \begin{array}{c} \uparrow \\ \mathbb{R}^{N \times N} \end{array} P = \text{softmax}(S), \quad \begin{array}{c} \mathbb{R}^{N \times d} \\ \swarrow \\ O = P V \leftarrow \mathbb{R}^{N \times d} \\ \uparrow \\ \mathbb{R}^{N \times N} \end{array} \end{array}$$

How about backward pass?

1. What is the “backward pass” of the attention mechanism?
 - a. E.g. of gradient through matmul
 - i. Given scalar loss L , and dO , what is dK , dQ , dV ?

How about backward pass?

1. What is the “backward pass” of the attention mechanism?

a. E.g. of gradient through matmul

$$O = PV$$

$$O_{ij} = \sum_k^N P_{ik} V_{kj} \Rightarrow$$

$$\underline{dO} := \frac{\partial L}{\partial O} \quad [\underline{dO}]_{ij} = \frac{\partial L}{\partial O_{ij}}$$

$$\frac{\partial L}{\partial V_{ij}} = \sum_k \frac{\partial L}{\partial O_{kj}} \cdot \frac{\partial O_{kj}}{\partial V_{ij}} = \sum_k \frac{\partial L}{\partial O_{kj}} \cdot P_{ki} = \sum_k P_{ki}^T \frac{\partial L}{\partial O_{kj}}$$

$$\underline{dV} := \frac{\partial L}{\partial V} \Rightarrow \underline{dV} = \underbrace{P^T}_{\in \mathbb{R}^{N \times N}} \underbrace{dO}_{\in \mathbb{R}^{N \times d}} \in \mathbb{R}^{N \times d}$$

$$\frac{\partial L}{\partial P_{ij}} = \sum_k \frac{\partial L}{\partial O_{ik}} \frac{\partial O_{ik}}{\partial P_{ij}} = \sum_k \frac{\partial L}{\partial O_{ik}} \cdot V_{jk} = \sum_k \frac{\partial L}{\partial O_{ik}} V_{jk}^T$$

$$\underline{dP} := \frac{\partial L}{\partial P} \Rightarrow \underline{dP} = \underbrace{dO}_{\in \mathbb{R}^{N \times d}} \underbrace{V^T}_{\in \mathbb{R}^{d \times N}} \in \mathbb{R}^{N \times N}$$

How about backward pass?

1. What is the “backward pass” of the attention mechanism?
 - a. E.g. gradient through softmax

1D case $\vec{x} \in \mathbb{R}^N$
 $y = \text{softmax}(\vec{x}) \in \mathbb{R}^N$

$$y = [y_1 \ y_2 \ \dots \ y_N]$$
$$y_i = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad \frac{\partial y_i}{\partial x_k} \begin{cases} y_i(1-y_k) & \text{if } i=k \\ -y_i y_k & \text{if } i \neq k \end{cases}$$
$$\frac{\partial L}{\partial x_i} = \sum_j \frac{\partial L}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i}$$
$$= \frac{\partial L}{\partial y_1} \cdot (-y_1 y_i) + \frac{\partial L}{\partial y_2} \cdot (-y_2 y_i) + \dots + \frac{\partial L}{\partial y_i} y_i - \frac{\partial L}{\partial y_i} y_i^2 + \dots$$
$$= y_i \left(\frac{\partial L}{\partial y_i} - \sum_j \frac{\partial L}{\partial y_j} \cdot y_j \right)$$

How about backward pass?

1. What is the “backward pass” of the attention mechanism?
 - a. E.g. gradient through softmax

$$P = \text{softmax}(S) \Rightarrow \text{softmax applied independently on each row!}$$

$$\underline{dP} := \frac{\partial L}{\partial P} \in \mathbb{R}^{N \times N}, \quad \underline{dS} := \frac{\partial L}{\partial S}$$

$$\Rightarrow [\underline{dS}]_{ij} = P_{ij} (dP_{ij} - \sum_k P_{ik} dP_{ik})$$

How about backward pass?

1. What is the “backward pass” of the attention mechanism?
 - a. E.g. gradient through softmax - nicer form (from paper)

Recall that $P_{i:} = \text{softmax}(S_{i:})$. Using the fact that the Jacobian of $y = \text{softmax}(x)$ is $\text{diag}(y) - yy^T$, we have that

$$dS_{i:} = (\text{diag}(P_{i:}) - P_{i:}P_{i:}^T)dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:})P_{i:},$$

How about backward pass?

1. What is the “backward pass” of the attention mechanism?

a. E.g. gradient through softmax - nicer form (from paper)

Recall that $P_{i:} = \text{softmax}(S_{i:})$. Using the fact that the Jacobian of $y = \text{softmax}(x)$ is $\text{diag}(y) - yy^T$, we have that

$$dS_{i:} = (\text{diag}(P_{i:}) - P_{i:}P_{i:}^T)dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:})P_{i:},$$

The subscript “i:” denotes the ith row

Pointwise multiplication

Standard Attention - Backward pass

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

- 1: Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
 - 2: Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dOV}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
 - 3: Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il}dP_{il})$, write $\mathbf{dS}$ to HBM.
 - 4: Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dSK}$, write $\mathbf{dQ}$ to HBM.
 - 5: Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
 - 6: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

Standard Attention - Backward pass

In forward pass, $\mathbf{S} = \mathbf{QK}^T$, $\mathbf{P} = \text{softmax}(\mathbf{S})$, $\mathbf{O} = \mathbf{PV}$

$$\mathbf{dO} := \frac{\partial L}{\partial \mathbf{O}}$$

Derivative of the loss wrt output
of the softmax

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

- 1: Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^T \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
 - 2: Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dOV}^T \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
 - 3: Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
 - 4: Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dSK}$, write $\mathbf{dQ}$ to HBM.
 - 5: Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^T \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
 - 6: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

Standard Attention - Backward pass

In forward pass, $\mathbf{S} = \mathbf{QK}^T$, $\mathbf{P} = \text{softmax}(\mathbf{S})$, $\mathbf{O} = \mathbf{PV}$

Flow through matmul

Algorithm 3 Standard Attention Backward Pass

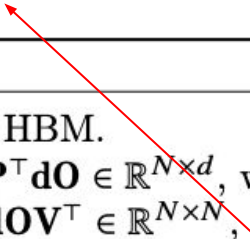
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

- 1: Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^T \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
 - 2: Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO} \mathbf{V}^T \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
 - 3: Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
 - 4: Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dS} \mathbf{K}$, write $\mathbf{dQ}$ to HBM.
 - 5: Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^T \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
 - 6: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

Standard Attention - Backward pass

In forward pass, $\mathbf{S} = \mathbf{QK}^T$, $\mathbf{P} = \text{softmax}(\mathbf{S})$, $\mathbf{O} = \mathbf{PV}$

Flow through softmax



Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

- 1: Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^T \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
 - 2: Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dOV}^T \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
 - 3: Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $\mathbf{dS}_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
 - 4: Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dSK}$, write $\mathbf{dQ}$ to HBM.
 - 5: Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^T \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
 - 6: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

Standard Attention - Backward pass

How many accesses to
HBM?

We first analyze the IO complexity of standard attention backward pass. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ reside in HBM, and at the end of the algorithm the outputs $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ are written to HBM.

At each step of the standard attention backward pass, one needs to load inputs of size Nd or N^2 from HBM, and needs to write the outputs of size N^2 or Nd to HBM. This incurs $\Theta(Nd + N^2)$ HBM accesses.

FlashAttention - Backwards pass

1. What have we already stored in HBM in the forward pass of FlashAttention?

FlashAttention - Backwards pass

1. What have we already stored in HBM in the forward pass of FlashAttention?

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

FlashAttention - Backwards pass

1. What have we already stored in HBM in the forward pass of FlashAttention?

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

The vectors m
and ℓ .



FlashAttention - Backwards pass

1. What was m and ℓ ?

FlashAttention - Backwards pass

1. What was m and ℓ ?

a. The i th element in m was the max element in the i th row of the matrix $S (=QK^T)$

FlashAttention - Backwards pass

1. What was m and ℓ ?

- a. The i th element in m was the max element in the i th row of the matrix $S (=QK^T)$
- b. The i th element in ℓ was the denominator of the softmax for the i th row in $S (=QK^T)$

$$L_i = \sum_j e^{q_i^T k_j}.$$

FlashAttention - Backwards pass

1. What is the “trick” in the backwards pass?
 - a. Just good old fusing of operations!
 - b. Start from dS_i :

$$dS_i = (\text{diag}(P_{i,:}) - P_{i,:}P_{i,:}^T)dP_{i,:} = P_{i,:} \circ dP_{i,:} - (P_{i,:}^T dP_{i,:})P_{i,:},$$

FlashAttention - Backwards pass

1. What is the “trick” in the backwards pass?

- a. Just good old fusing of operations!
- b. Start from dS_i :

$$dS_i = (\text{diag}(P_{i,:}) - P_{i,:}P_{i,:}^T)dP_{i,:} = P_{i,:} \circ dP_{i,:} - \boxed{(P_{i,:}^T dP_{i,:})}P_{i,:},$$

Expand this out,
remembering $dP_{ij} = do_i^T v_j$.

FlashAttention - Backwards pass

1. What is the “trick” in the backwards pass?
 - a. Just good old fusing of operations!
 - b. Start from dS_i :

Define

$$D_i = P_i^T dP_i = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j = do_i^T o_i,$$

then

$$dS_i = P_i \circ dP_i - D_i P_i.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij} (dP_{ij} - D_i).$$

FlashAttention - Backwards pass

1. What is the “trick” in the backwards pass?

- Just good old fusing of operations!
- Start from dS_i :

This is a scalar, and you can loop over only d elements in do_i and o_i as opposed to the N elements in P_i^T , dP_i !!!

Define

$$D_i = P_i^T dP_i = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j = \boxed{do_i^T o_i},$$

then

$$dS_i = P_i \circ dP_i - D_i P_i.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij} (dP_{ij} - D_i).$$

FlashAttention - Backwards pass

1. What is the “trick” in the backwards pass?
 - a. Just good old fusing of operations!
 - b. Start from dS_i .
 - c. Everything else just works out!

Now we can get the gradients $d\mathbf{Q}$ and $d\mathbf{K}$. Recall that $S_{ij} = q_i^T k_j$, so

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij} (dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) k_j.$$

Similarly,

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij} (dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) q_i.$$

FlashAttention - Backwards pass

1. Actual algorithm is “just” a block version of the equations we just saw!

FlashAttention - Backwards pass

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} , pseudo-random number generator state $\mathcal{R}$ from the forward pass.

- 1: Set the pseudo-random number generator state to $\mathcal{R}$.
 - 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
 - 9: **for** $1 \leq i \leq T_r$ **do**
 - 10: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 11: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 12: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
 - 13: On chip, compute $\mathbf{P}_{ij} = \text{diag}(\ell_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 14: On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop} .
 - 15: On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
 - 16: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_{ij}^{\text{dropped}})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
 - 17: On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 18: On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
 - 19: On chip, compute $\mathbf{D}_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
 - 20: On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - \mathbf{D}_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 21: Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
 - 22: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \tau \mathbf{dS}_{ij}^T \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
 - 23: **end for**
 - 24: Write $\mathbf{dK}_j, \mathbf{dV}_j \leftarrow \mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
 - 25: **end for**
 - 26: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

FlashAttention - Backwards pass

1. How many HBM memory accesses?
 - a. Same as in forward pass!

$$\Theta(NdT_c) = \Theta\left(\frac{N^2d^2}{M}\right).$$

FlashAttention - Backwards pass

1. How does it handle dropout?
 - a. Don't you need to store the N^2 mask used in the forward pass?

FlashAttention - Backwards pass

1. How does it handle dropout?

- a. Don't you need to store the N^2 mask used in the forward pass?
- b. No!

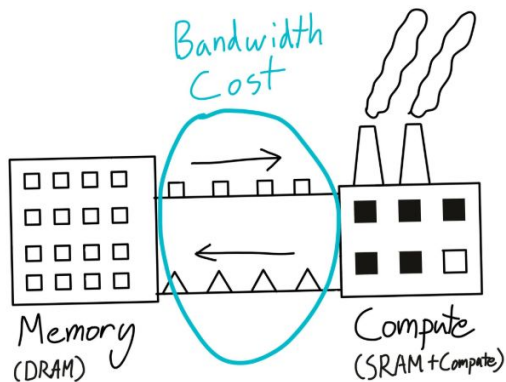
1. We do not need to store the dropout mask of size $O(N^2)$ from the forward pass. Instead, we can save the pseudo-random number generator states from the forward pass and re-generate the dropout mask in the backward pass. This allows us to only use $O(N)$ extra memory.

Outline

1. The basics
2. FlashAttention forward pass
3. FlashAttention backward pass
4. Brief notes on FlashAttention-2+3

FlashAttention-2

1. Taking the analogy further - the factory has different kinds of “machines” (compute units) that can process the “materials” (data)
 - a. In GPUs, the “machines” for matmul are hypertuned and hyperefficient, while other “machines” (such as that for arithmetic operations) are not as good



FlashAttention-2

1. Forward pass + backward pass - big-theta accesses to HBM remains the same, but change the operations (the order of the operations + the actual operations) on the SRAM to minimise number of non-matmul operations

FlashAttention-2

1. Forward pass + backward pass - big-theta accesses to HBM remains the same, but change the operations (the order of the operations + the actual operations) on the SRAM to minimise number of non-matmul operations
 - a. For forward pass:

1. We do not have to rescale both terms of the output update by $\text{diag}(\ell^{(2)})^{-1}$:

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - \mathbf{m}^{(2)}} \mathbf{V}^{(2)}.$$

We can instead maintain an “un-scaled” version of $\mathbf{O}^{(2)}$ and keep around the statistics $\ell^{(2)}$:

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(\ell^{(1)})^{-1} \mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)} - \mathbf{m}^{(2)}} \mathbf{V}^{(2)}.$$

Only at the every end of the loop do we scale the final $\tilde{\mathbf{O}}^{(\text{last})}$ by $\text{diag}(\ell^{(\text{last})})^{-1}$ to get the right output.

2. We do not have to save both the max $\mathbf{m}^{(j)}$ and the sum of exponentials $\ell^{(j)}$ for the backward pass. We only need to store the logsumexp $L^{(j)} = \mathbf{m}^{(j)} + \log(\ell^{(j)})$.

FlashAttention-2

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

1. We do not have to rescale both terms of the output update by $\text{diag}(\ell^{(2)})^{-1}$:

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{s}^{(2)} - m^{(2)}} \mathbf{V}^{(2)}.$$

FlashAttention-2

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$
 (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L .
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

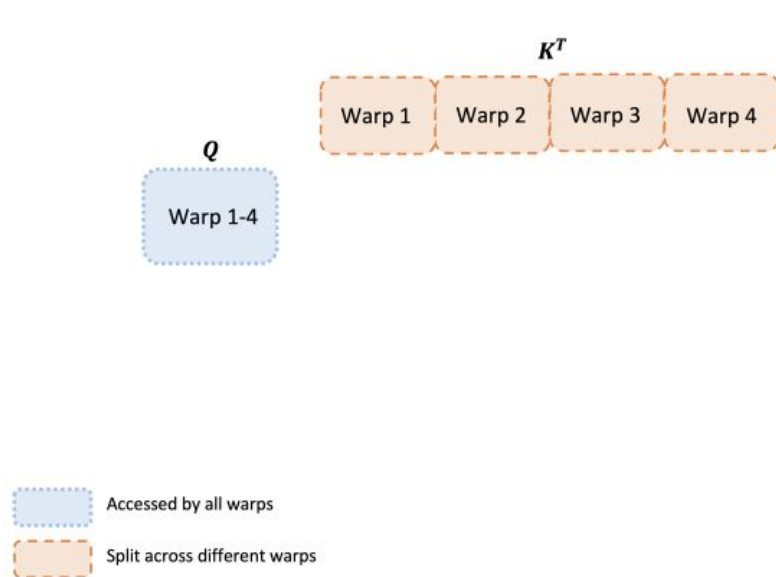
FlashAttention-2

1. Forward pass + backward pass - big-theta accesses to HBM remains the same, but change the operations (the order of the operations + the actual operations) on the SRAM to minimise number of non-matmul operations
 - a. For backward pass:

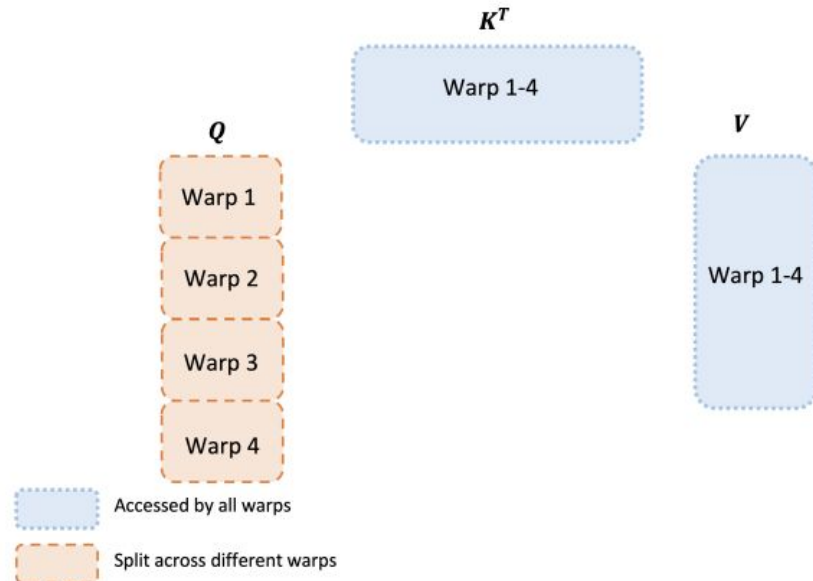
The backward pass of FLASHATTENTION-2 is almost the same as that of FLASHATTENTION. We make a minor tweak to only use the row-wise logsumexp L instead of both the row-wise max and row-wise sum of exponentials in the softmax. We include the backward pass description in Algorithm 2 for completeness.

FlashAttention-2

1. Other scheduling tricks!



(a) FLASHATTENTION



(b) FLASHATTENTION-2

FlashAttention-3: Even more tricks

1. [Blog](#) and [paper](#)
2. Exploits newer features in NVIDIA's Hopper architecture for faster attention computations
 - a. WGMMA (Warpgroup Matrix Multiply-Accumulate)
 - b. TMA (Tensor Memory Accelerator)
 - c. Variable length sequences, persistent kernel, in-kernel transpose for FP8, and more...
3. Refer to H100 [white paper](#)

The best news

1. [pytorch/transformers](#) library automatically uses the latest FlashAttention (if your sys/env can support it) without you having to do anything about it
2. Easier than ever to write your own kernels using [Triton](#)
 - a. You don't have to write things in non-Python(ic) languages!

End!